Contents

Dynamic Programming - Comprehensive Study Guide (v2) **2**
Design and Analysis of Algorithms (Homework 2) . . . 2
Document Structure . . . 2
Table of Contents . . . 2

Core Homework Problems (1-8) **3**
1. Triangle Path Maximum Sum . . . 3
2. Coin Change Problem . . . 7
3. Matrix Chain Multiplication . . . 11
4. Edit Distance . . . 17
5. Longest Palindromic Subsequence . . . 24
6. Shortest Common Supersequence . . . 29
7. Beautiful Sequence (Minimum Additions for Palindrome) . . . 35
8. Subset Sum Problem . . . 40

Advanced Problems and Classic DP (9-26) **45**
9. 0/1 Knapsack Problem . . . 45
10. Longest Increasing Subsequence (LIS) . . . 46
11. Rod Cutting Problem . . . 48
12. Maximum Subarray Sum (Kadane's Algorithm) . . . 50
13. Climbing Stairs . . . 51
14. House Robber . . . 52
15. Coin Change (Minimum Coins) . . . 54
16. Word Break . . . 55
17. Partition Problem . . . 56
18. Egg Drop Problem . . . 58
19. Longest Common Substring . . . 59
20. Palindrome Partitioning (Minimum Cuts) . . . 61
21. Optimal Binary Search Tree . . . 63
22. Matrix Chain Multiplication . . . 65
23. Longest Palindromic Substring . . . 65
24. Minimum Path Sum in Grid . . . 67
25. Unique Paths in Grid . . . 70
26. Interleaving String . . . 71

Quick Reference **74**
Complexity Summary . . . 74
Pattern Recognition Guide . . . 75
Master Theorem Cheat Sheet . . . 75
Key Formulas . . . 76
Optimization Techniques . . . 76
Proof Templates . . . 76
Quiz Strategy . . . 77
Final Checklist . . . 77

Dynamic Programming - Comprehensive Study Guide (v2)

Design and Analysis of Algorithms (Homework 2)

Structured Format: Every Problem Has All Components

Document Structure

Each problem follows this consistent format:

1. **Problem Statement** — Clear definition with examples
2. **Worked Example** — Step-by-step solution with visualization

3. **Recurrence Relation** — Mathematical formulation with base cases
4. **Algorithm** — Pseudocode or Python implementation
5. **Time Complexity Analysis** — Big-O with recurrence proof
6. **Correctness Proof** — Formal proof by induction

Table of Contents

Core Homework Problems (1-8)

1. Triangle Path Maximum Sum
2. Coin Change Problem
3. Matrix Chain Multiplication
4. Edit Distance
5. Longest Palindromic Subsequence
6. Shortest Common Supersequence
7. Beautiful Sequence
8. Subset Sum Problem

Advanced DP Problems (9-18)

9. 0/1 Knapsack Problem
10. Longest Increasing Subsequence (LIS)
11. Rod Cutting Problem
12. Maximum Subarray Sum (Kadane's Algorithm)
13. Climbing Stairs
14. House Robber
15. Coin Change (Minimum Coins)
16. Word Break
17. Partition Problem
18. Egg Drop Problem

Classic DP Problems (19-26)

19. Longest Common Substring
20. Palindrome Partitioning (Minimum Cuts)
21. Optimal Binary Search Tree
22. Matrix Chain Multiplication

23. Longest Palindromic Substring
24. Minimum Path Sum in Grid
25. Unique Paths in Grid
26. Interleaving String

Core Homework Problems (1-8)

---

1. Triangle Path Maximum Sum

Problem Statement

Given a triangle represented as nested arrays, find the maximum path sum from top to bottom. You can move to adjacent numbers on the row below (from position j to either j or $j+1$).

Input Format:

```
triangle = [
    [3],
   [10, 7],
  [18, 4, 13],
 [8, 5, 9, 3]
]
```

Output: Maximum path sum = 23 (path: $3 \to 7 \to 4 \to 9$)

Constraints: - $1 \leq n \leq 200$ (number of rows) - Adjacent means position j in row i connects to positions j and $j+1$ in row $i+1$

Worked Example: Step-by-Step

Given Triangle:

```
     3
   10  7
  18  4 13
  8  5  9  3
```

Strategy: Build from top to bottom, storing maximum path sum to reach each position.

Step 1: Initialize Row 0

`dp[0][0] = 3`

Step 2: Compute Row 1 - `dp[1][0] = triangle[1][0] + dp[0][0] = 10 + 3 = 13` (Can only come from position 0 in row 0) - `dp[1][1] = triangle[1][1] + dp[0][0] = 7 + 3 = 10` (Can only come from position 0 in row 0)

DP Table after Row 1:

```
     3
   13 10
```

Step 3: Compute Row 2 - `dp[2][0] = triangle[2][0] + dp[1][0] = 18 + 13 = 31`
(Can only come from position 0 in row 1) - `dp[2][1] = triangle[2][1] + max(dp[1][0], dp[1][1]) = 4 + max(13, 10) = 4 + 13 = 17`
(Can come from position 0 or 1 in row 1, choose max) - `dp[2][2] = triangle[2][2] + dp[1][1] = 13 + 10 = 23`
(Can only come from position 1 in row 1)

DP Table after Row 2:

```
     3
   13 10
  31 17 23
```

Step 4: Compute Row 3 - `dp[3][0] = triangle[3][0] + dp[2][0] = 8 + 31 = 39` - `dp[3][1] = triangle[3][1] + max(dp[2][0], dp[2][1]) = 5 + max(31, 17) = 5 + 31 = 36` - `dp[3][2] = triangle[3][2] + max(dp[2][1], dp[2][2]) = 9 + max(17, 23) = 9 + 23 = 32` - `dp[3][3] = triangle[3][3] + dp[2][2] = 3 + 23 = 26`

Final DP Table:

```
     3
   13 10
  31 17 23
  39 36 32 26
```

Answer: `max(39, 36, 32, 26) = 39`

Path Reconstruction: `3 → 10 → 18 → 8` (sum = 39)

Recurrence Relation

State Definition:
Let $dp[i][j]$ = maximum path sum from top ($row0$, $position0$) to position (i, j)

Recurrence:

$$dp[i][j] = triangle[i][j] + \begin{cases} dp[i-1][0] & \text{if } j = 0 \\ dp[i-1][i-1] & \text{if } j = i \\ \max(dp[i-1][j-1], dp[i-1][j]) & \text{otherwise} \end{cases}$$

Base Case:

$$dp[0][0] = triangle[0][0]$$

Final Answer:

$$\max_{j=0}^{n-1} dp[n-1][j]$$

Algorithm

```python
def maximumTotal(triangle):
    n = len(triangle)
```

```python
    # Create DP table
    dp = [[0] * (i + 1) for i in range(n)]

    # Base case
    dp[0][0] = triangle[0][0]

    # Fill row by row
    for i in range(1, n):
        for j in range(i + 1):
            if j == 0:
                # Leftmost: can only come from dp[i-1][0]
                dp[i][j] = triangle[i][j] + dp[i-1][0]
            elif j == i:
                # Rightmost: can only come from dp[i-1][i-1]
                dp[i][j] = triangle[i][j] + dp[i-1][i-1]
            else:
                # Middle: take max of two parents
                dp[i][j] = triangle[i][j] + max(dp[i-1][j-1], dp[i-1][j])

    # Return maximum value in last row
    return max(dp[n-1])

# Space-optimized version (O(n) space)
def maximumTotal_optimized(triangle):
    n = len(triangle)
    dp = triangle[-1][:]  # Start from bottom row

    # Work backwards
    for row in range(n-2, -1, -1):
        for col in range(len(triangle[row])):
            dp[col] = triangle[row][col] + max(dp[col], dp[col+1])

    return dp[0]
```

Time Complexity Analysis

Intuitive Analysis: - Process each cell in the triangle exactly once - Row 0 has 1 cell, Row 1 has 2 cells, ..., Row $n-1$ has n cells - Total cells $= 1+2+3+\cdots+n = \frac{n(n+1)}{2}$ - Each cell takes $O(1)$ time (one addition and one max operation) - **Total Time:** $O(n^2)$

Formal Recurrence:

Let $T(n)$ = time to compute all DP entries for triangle with n rows.

Recurrence Setup:

$$T(n) = \sum_{i=0}^{n-1}(i+1) \cdot O(1)$$

The sum $\sum_{i=0}^{n-1}(i+1)$ counts the total number of cells:

$$\sum_{i=0}^{n-1}(i+1) = \sum_{k=1}^{n} k = \frac{n(n+1)}{2}$$

Therefore:

$$T(n) = O\left(\frac{n(n+1)}{2}\right) = O(n^2)$$

Verification by Induction:

Claim: $T(n) = cn^2$ for some constant $c > 0$.

Base Case: $n = 1$
Process 1 cell in $O(1)$ time. $T(1) = c \cdot 1^2 = c$

Inductive Step:
Assume $T(k) = ck^2$ for all $k < n$.

For triangle with n rows, we process: - All rows 0 to $n-2$: takes $T(n-1) = c(n-1)^2$ by IH - Row $n-1$: takes $n \cdot O(1) = cn$

Total:

$$T(n) = c(n-1)^2 + cn = c(n^2 - 2n + 1 + n) = c(n^2 - n + 1) = O(n^2)$$

Space Complexity: $O(n^2)$ for 2D table, or $O(n)$ with space optimization.

Correctness Proof

Claim: After processing row i, $dp[i][j]$ contains the maximum path sum from the top to position (i, j).

Proof by Strong Induction on row number i:

Base Case: $i = 0$
$dp[0][0] = triangle[0][0]$. This is the only position in row 0, and the only path is to stay there. Correct

Inductive Hypothesis (IH):
Assume for all rows $k < i$, $dp[k][j]$ correctly contains the maximum path sum to position (k, j) for all valid j.

Inductive Step:
Show $dp[i][j]$ is correct for row i.

Consider position (i, j) where $0 \leq j \leq i$. To reach (i, j), we must come from row $i-1$.

Case 1: $j = 0$ **(leftmost position)**
Can only come from $(i-1, 0)$.

By IH, $dp[i-1][0]$ is the maximum path sum to $(i-1, 0)$.

Algorithm computes: $dp[i][0] = triangle[i][0] + dp[i-1][0]$

This is correct: extend the best path to $(i-1, 0)$ by adding $triangle[i][0]$

Case 2: $j = i$ **(rightmost position)**
Can only come from $(i-1, i-1)$.

By IH, $dp[i-1][i-1]$ is correct.

Algorithm computes: $dp[i][i] = triangle[i][i] + dp[i-1][i-1]$

This is correct

Case 3: $0 < j < i$ **(middle positions)**
Can come from either: - $(i-1, j-1)$ — diagonal-left parent - $(i-1, j)$ — diagonal-right parent

By IH, both $dp[i-1][j-1]$ and $dp[i-1][j]$ are correct.

Any path to (i, j) must come from one of these parents. The maximum path chooses the parent with the larger value:

Algorithm computes: $dp[i][j] = triangle[i][j] + \max(dp[i-1][j-1], dp[i-1][j])$

Optimal substructure: If the optimal path to (i, j) comes through parent p, then the path from top to p must also be optimal (otherwise we could substitute a better path to p and improve the total).

Since we correctly have the maximum path sums to both parents (by IH), taking the max gives the maximum path sum to (i, j)

Conclusion:
By strong induction, $dp[i][j]$ is correct for all positions. Therefore, $\max_j dp[n-1][j]$ correctly gives the maximum path sum from top to bottom. □

---

2. Coin Change Problem

Problem Statement

Given: - Coin denominations: $C = \{c_1, c_2, \dots, c_n\}$ (each with unlimited supply) - Target value: V

Find: Minimum number of coins needed to make value V.

Example 1:

```
C = {1, 3, 4}, V = 6
Output: 2 coins (3 + 3)
```

Example 2:

```
C = {2, 5}, V = 3
Output: -1 (impossible)
```

Example 3:

```
C = {1, 2, 5}, V = 11
Output: 3 coins (5 + 5 + 1)
```

Worked Example: Step-by-Step

Given: $C = \{1, 3, 4\}$, $V = 6$

Goal: Build up solutions from value 0 to value 6.

Initial State:

```
dp[0] = 0  (0 coins needed to make value 0)
dp[1..6] = ∞  (unknown initially)
```

Step 1: Compute dp[1]

Try each coin: - Coin 1: Can use it ($1 \leq 1$). Cost $= 1 + dp[1-1] = 1 + dp[0] = 1 + 0 = 1$ - Coin 3: Cannot use ($3 > 1$) - Coin 4: Cannot use ($4 > 1$)

Result: $dp[1] = 1$ (use one coin of denomination 1)

Step 2: Compute dp[2]

Try each coin: - Coin 1: $1 + dp[2-1] = 1 + dp[1] = 1 + 1 = 2$ - Coin 3: Cannot use ($3 > 2$) - Coin 4: Cannot use ($4 > 2$)

Result: $dp[2] = 2$ (use $1 + 1$)

Step 3: Compute dp[3]

Try each coin: - Coin 1: $1 + dp[3-1] = 1 + dp[2] = 1 + 2 = 3$ - Coin 3: $1 + dp[3-3] = 1 + dp[0] = 1 + 0 = 1$ $\leftarrow$ **Best!** - Coin 4: Cannot use ($4 > 3$)

Result: $dp[3] = 1$ (use one coin of denomination 3)

Step 4: Compute dp[4]

Try each coin: - Coin 1: $1 + dp[3] = 1 + 1 = 2$ - Coin 3: $1 + dp[1] = 1 + 1 = 2$ - Coin 4: $1 + dp[0] = 1 + 0 = 1$ $\leftarrow$ **Best!**

Result: $dp[4] = 1$ (use one coin of denomination 4)

Step 5: Compute dp[5]

Try each coin: - Coin 1: $1 + dp[4] = 1 + 1 = 2$ $\leftarrow$ **Best!** - Coin 3: $1 + dp[2] = 1 + 2 = 3$ - Coin 4: $1 + dp[1] = 1 + 1 = 2$

Result: $dp[5] = 2$ (use $4 + 1$ or $3 + 1 + 1$, both take 2 coins)

Step 6: Compute dp[6]

Try each coin: - Coin 1: $1 + dp[5] = 1 + 2 = 3$ - Coin 3: $1 + dp[3] = 1 + 1 = 2$ $\leftarrow$ **Best!** - Coin 4: $1 + dp[2] = 1 + 2 = 3$

Result: $dp[6] = 2$ (use $3 + 3$)

Final DP Table: | Value | 0 | 1 | 2 | 3 | 4 | 5 | 6 | |——-|—|—|—|—|—|—|—| | Min Coins | 0 | 1 | 2 | 1 | 1 | 2 | **2** |

Answer: 2 coins needed (combination: $3 + 3 = 6$)

Recurrence Relation

State Definition:
Let $dp[v]$ = minimum number of coins needed to make value v

Recurrence:

$$dp[v] = \begin{cases} 0 & \text{if } v = 0 \\ \infty & \text{if } v < 0 \\ \min_{c_i \in C, c_i \leq v}\{1 + dp[v - c_i]\} & \text{otherwise} \end{cases}$$

Explanation: - To make value v, try using each coin c_i (where $c_i \leq v$) - If we use coin c_i, we need 1 coin plus the minimum coins for remaining value $(v - c_i)$ - Take the minimum over all valid coin choices

Final Answer:

$$\text{If } dp[V] = \infty, \text{ no solution exists. Otherwise, answer is } dp[V]$$

Algorithm

```python
def coinChange(coins, amount):
    """
    Returns minimum number of coins needed to make amount.
    Returns -1 if impossible.
    """
    # Initialize DP array
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0  # Base case

    # Build up solutions for values 1 to amount
    for value in range(1, amount + 1):
        # Try each coin
        for coin in coins:
            if coin <= value and dp[value - coin] != float('inf'):
                dp[value] = min(dp[value], 1 + dp[value - coin])

    # Return answer
    return dp[amount] if dp[amount] != float('inf') else -1

# Alternative: With path reconstruction
def coinChangeWithPath(coins, amount):
    dp = [float('inf')] * (amount + 1)
    parent = [-1] * (amount + 1)
    dp[0] = 0

    for value in range(1, amount + 1):
        for coin in coins:
            if coin <= value and dp[value - coin] + 1 < dp[value]:
                dp[value] = dp[value - coin] + 1
                parent[value] = coin

    if dp[amount] == float('inf'):
        return -1, []

    # Reconstruct path
    path = []
    v = amount
    while v > 0:
        coin_used = parent[v]
        path.append(coin_used)
        v -= coin_used
```

```
    return dp[amount], path
```

Time Complexity Analysis

Intuitive Analysis: - **Outer loop:** Iterate through values 1 to $V \to V$ iterations - **Inner loop:** For each value, try all n coins $\to n$ iterations - **Each trial:** Constant time $O(1)$ (array lookup, addition, comparison) - **Total:** $O(V \cdot n)$

Space: $O(V)$ for the DP array

Formal Recurrence:

Let $T(V)$ = time to solve coin change for target value V with n coins.

Analysis of algorithm:

```
for v = 1 to V:              // V iterations
    for c in coins:          // n iterations
        O(1) operations      // constant work
```

Recurrence:

$$T(V) = \sum_{v=1}^{V} n \cdot O(1) = V \cdot n \cdot O(1) = O(Vn)$$

Verification:
Each value from 1 to V is processed exactly once. For each value, we examine all n coins. Each coin examination involves: - Check if $coin \leq value$: $O(1)$ - Array access $dp[value - coin]$: $O(1)$ - Comparison and min: $O(1)$

Total operations: $V \times n \times O(1) = O(Vn)$

Alternative Recursive Formulation (without DP):

$$T(v) = \begin{cases} O(1) & \text{if } v = 0 \\ \sum_{i=1}^{n} T(v - c_i) & \text{otherwise} \end{cases}$$

This gives exponential time $O(n^V)$ without memoization, because same subproblems are solved repeatedly.

With DP/Memoization:
Each of V subproblems is solved exactly once, and each takes $O(n)$ time to try all coins.
Total: $O(Vn)$

Correctness Proof

Claim: After processing value v, $dp[v]$ contains the minimum number of coins needed to make value v, or ∞ if impossible.

Proof by Strong Induction on the value v:

Base Case: $v = 0$
$dp[0] = 0$ is correct. Zero coins are needed to make value 0

Inductive Hypothesis (IH):
Assume $dp[k]$ is correct for all k where $0 \leq k < v$.

Inductive Step:
Show $dp[v]$ is correct when we compute it.

When we compute $dp[v]$:

```
for coin in coins:
    if coin <= v:
        dp[v] = min(dp[v], 1 + dp[v - coin])
```

Case 1: No valid solution exists for value v
If it's impossible to make value v, then for every coin $c_i \in C$ either: - $c_i > v$, or - $dp[v - c_i] = \infty$ (impossible to make $v - c_i$)

In both cases, $dp[v]$ remains ∞, which is correct

Case 2: A valid solution exists for value v
Any solution to make value v must use at least one coin. Say the optimal solution uses coin c_j as one of the coins. Then:

$$\text{MinCoins}(v) = 1 + \text{MinCoins}(v - c_j)$$

By **IH**, $dp[v - c_j]$ is correct (since $v - c_j < v$).

Our algorithm tries all possible coins:

$$dp[v] = \min_{c_i \in C, c_i \leq v} \{1 + dp[v - c_i]\}$$

Since we try coin c_j among all coins, and all $dp[v - c_i]$ values are correct by IH, we get:

$$dp[v] = 1 + dp[v - c_j] = 1 + \text{MinCoins}(v - c_j) = \text{MinCoins}(v)$$

This is correct

Optimal Substructure Justification:
If the optimal solution for value v uses coin c_j, then the remaining value $(v - c_j)$ must be made with the minimum number of coins. If not, we could substitute a better solution for $(v - c_j)$ and reduce the total coin count, contradicting optimality.

Conclusion:
By strong induction on v, $dp[v]$ is correct for all values. Therefore, $dp[V]$ correctly gives the minimum number of coins needed to make value V (or ∞ if impossible). $\square$

---

3. Matrix Chain Multiplication

Problem Statement

Given a chain of n matrices to multiply: $A_1 \times A_2 \times \cdots \times A_n$

Each matrix A_i has dimensions $p_{i-1} \times p_i$.

Goal: Find the optimal parenthesization that minimizes the total number of scalar multiplications.

Note: Matrix multiplication is associative but not commutative. $(AB)C = A(BC)$ but $AB \neq BA$ in general.

Cost of multiplying two matrices:
Multiplying matrix A of size $p \times q$ with matrix B of size $q \times r$ costs $p \times q \times r$ scalar multiplications and produces a matrix of size $p \times r$.

Example:

```
Given dimensions: [10, 20, 30, 40, 30]
This represents 4 matrices:
- A1: 10 × 20
- A2: 20 × 30
- A3: 30 × 40
- A4: 40 × 30

Two different parenthesizations:
1. ((A1×A2)×A3)×A4 = 26,000 operations
2. A1×(A2×(A3×A4)) = 30,000 operations

Optimal: (A1×(A2×A3))×A4 = 15,200 operations
```

Worked Example: Step-by-Step

Given: Dimensions array $p = [10, 20, 30, 40, 30]$
This means: - A_1: 10×20 - A_2: 20×30 - A_3: 30×40 - A_4: 40×30

Goal: Minimize the cost of computing $A_1 \times A_2 \times A_3 \times A_4$

Strategy: Build DP table for increasing chain lengths.

DP Table: $dp[i][j]$ = minimum cost to multiply matrices A_i through A_j

---

Step 1: Base Case (Chain Length = 1)

Single matrices have no multiplication cost.

```
dp[1][1] = 0
dp[2][2] = 0
dp[3][3] = 0
dp[4][4] = 0
```

Step 2: Chain Length = 2

dp[1][2]: Cost of $A_1 \times A_2$
Only one way to multiply: $(A_1) \times (A_2)$ - Dimensions: $(10 \times 20) \times (20 \times 30)$ - Cost: $10 \times 20 \times 30 = 6000$ - $dp[1][2] = 6000$

dp[2][3]: Cost of $A_2 \times A_3$
- Dimensions: $(20 \times 30) \times (30 \times 40)$ - Cost: $20 \times 30 \times 40 = 24000$ - $dp[2][3] = 24000$

dp[3][4]: Cost of $A_3 \times A_4$
- Dimensions: $(30 \times 40) \times (40 \times 30)$ - Cost: $30 \times 40 \times 30 = 36000$ - $dp[3][4] = 36000$

Step 3: Chain Length = 3

dp[1][3]: Cost of $A_1 \times A_2 \times A_3$

Try all split points:

Split at k=1: $(A_1) \times (A_2 \times A_3)$ - Cost of left part: $dp[1][1] = 0$ - Cost of right part: $dp[2][3] = 24000$ - Cost to combine: $p[0] \times p[1] \times p[3] = 10 \times 20 \times 40 = 8000$ - **Total:** $0 + 24000 + 8000 = 32000$

Split at k=2: $(A_1 \times A_2) \times (A_3)$ - Cost of left part: $dp[1][2] = 6000$ - Cost of right part: $dp[3][3] = 0$ - Cost to combine: $p[0] \times p[2] \times p[3] = 10 \times 30 \times 40 = 12000$ - **Total:** $6000 + 0 + 12000 = 18000 \leftarrow$ **Better!**

Result: $dp[1][3] = 18000$ (split at k=2)

---

dp[2][4]: Cost of $A_2 \times A_3 \times A_4$

Split at k=2: $(A_2) \times (A_3 \times A_4)$ - Left: $dp[2][2] = 0$ - Right: $dp[3][4] = 36000$ - Combine: $20 \times 30 \times 30 = 18000$ - **Total:** $0 + 36000 + 18000 = 54000$

Split at k=3: $(A_2 \times A_3) \times (A_4)$ - Left: $dp[2][3] = 24000$ - Right: $dp[4][4] = 0$ - Combine: $20 \times 40 \times 30 = 24000$ - **Total:** $24000 + 0 + 24000 = 48000 \leftarrow$ **Better!**

Result: $dp[2][4] = 48000$ (split at k=3)

Step 4: Chain Length = 4

dp[1][4]: Cost of $A_1 \times A_2 \times A_3 \times A_4$

Split at k=1: $(A_1) \times (A_2 \times A_3 \times A_4)$ - Left: $dp[1][1] = 0$ - Right: $dp[2][4] = 48000$ - Combine: $10 \times 20 \times 30 = 6000$ - **Total:** $0 + 48000 + 6000 = 54000$

Split at k=2: $(A_1 \times A_2) \times (A_3 \times A_4)$ - Left: $dp[1][2] = 6000$ - Right: $dp[3][4] = 36000$ - Combine: $10 \times 30 \times 30 = 9000$ - **Total:** $6000 + 36000 + 9000 = 51000$

Split at k=3: $(A_1 \times A_2 \times A_3) \times (A_4)$ - Left: $dp[1][3] = 18000$ - Right: $dp[4][4] = 0$ - Combine: $10 \times 40 \times 30 = 12000$ - **Total:** $18000 + 0 + 12000 = 30000 \leftarrow$ **Best!**

Result: $dp[1][4] = 30000$ (split at k=3)

Final DP Table:

	1	2	3	4
1	0	6000	18000	**30000**
2		0	24000	48000
3			0	36000
4				0

Answer: 30,000 scalar multiplications

Optimal Parenthesization: $(A_1 \times A_2 \times A_3) \times A_4$
Or more explicitly: $((A_1 \times A_2) \times A_3) \times A_4$

Recurrence Relation

State Definition:
Let $dp[i][j]$ = minimum number of scalar multiplications needed to compute $A_i \times A_{i+1} \times \cdots \times A_j$

Recurrence:

$$dp[i][j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \le k < j}\{dp[i][k] + dp[k+1][j] + p_{i-1} \cdot p_k \cdot p_j\} & \text{if } i < j \end{cases}$$

Explanation: - **Base case:** Single matrix requires no multiplications - **Recursive case:** To multiply $A_i \cdots A_j$, split at position k: - Multiply left part: $A_i \cdots A_k$ (cost: $dp[i][k]$) - Multiply right part: $A_{k+1} \cdots A_j$ (cost: $dp[k+1][j]$) - Multiply the two results: $(p_{i-1} \times p_k) \times (p_k \times p_j)$ costs $p_{i-1} \cdot p_k \cdot p_j$ - Try all possible split points k and take minimum

Input: Array p of size $n+1$ where matrix A_i has dimensions $p_{i-1} \times p_i$

Output: $dp[1][n]$ gives minimum cost for entire chain

Algorithm

```python
def matrixChainOrder(p):
    """
    p: dimensions array of size n+1
    Returns: minimum number of multiplications
    """
    n = len(p) - 1  # number of matrices

    # Create DP table (indexed from 1 to n)
    dp = [[0] * (n + 1) for _ in range(n + 1)]
    split = [[0] * (n + 1) for _ in range(n + 1)]  # for reconstruction

    # Base case: single matrices (length = 1)
    # dp[i][i] = 0 (already initialized)

    # Fill table for increasing chain lengths
    for length in range(2, n + 1):  # chain length
        for i in range(1, n - length + 2):  # starting position
            j = i + length - 1  # ending position
            dp[i][j] = float('inf')

            # Try all split points
            for k in range(i, j):
                # Cost of (A_i...A_k) × (A_{k+1}...A_j)
                cost = dp[i][k] + dp[k+1][j] + p[i-1] * p[k] * p[j]

                if cost < dp[i][j]:
```

```python
                    dp[i][j] = cost
                    split[i][j] = k  # remember split point

    return dp[1][n], split

def printOptimalParens(split, i, j):
    """Reconstruct and print optimal parenthesization"""
    if i == j:
        print(f"A{i}", end="")
    else:
        print("(", end="")
        printOptimalParens(split, i, split[i][j])
        printOptimalParens(split, split[i][j] + 1, j)
        print(")", end="")

# Example usage
p = [10, 20, 30, 40, 30]
min_cost, split = matrixChainOrder(p)
print(f"Minimum cost: {min_cost}")
print("Optimal parenthesization: ", end="")
printOptimalParens(split, 1, len(p) - 1)
print()
```

Time Complexity Analysis

Intuitive Analysis: - **Outer loop (length):** Runs from 2 to $n \to O(n)$ iterations - **Middle loop (starting position i):** For each length, runs $O(n)$ times - **Inner loop (split point k):** For each (i, j) pair, runs $O(n)$ times - **Total:** $O(n) \times O(n) \times O(n) = O(n^3)$

Space Complexity: $O(n^2)$ for the DP table

Formal Recurrence:

Let $T(n)$ = time to solve matrix chain multiplication for n matrices.

Analysis:

```
for length = 2 to n:                  // n-1 iterations
    for i = 1 to n-length+1:          //   n iterations
        for k = i to j-1:             //   n iterations
            O(1) operations
```

Total cells to fill: Number of entries where $i < j$ in an $n \times n$ table $= \frac{n(n-1)}{2} = O(n^2)$

Work per cell: For cell (i, j) where $j - i = \ell$, we try ℓ split points. On average, $\ell \approx n/2$.

Total work:

$$T(n) = \sum_{\ell=1}^{n-1} \sum_{i=1}^{n-\ell} \ell \cdot O(1)$$

For each chain length ℓ, there are approximately n starting positions, and each requires ℓ operations:

$$T(n) = \sum_{\ell=1}^{n-1} n \cdot \ell = n \sum_{\ell=1}^{n-1} \ell = n \cdot \frac{(n-1)n}{2} = O(n^3)$$

Verification: - Total subproblems: $O(n^2)$ (all pairs (i, j) where $i \leq j$) - Work per subproblem: $O(n)$ (trying all k split points) - **Total:** $O(n^2) \times O(n) = O(n^3)$

Correctness Proof

Claim: After processing all chains of length ℓ, $dp[i][j]$ contains the minimum number of scalar multiplications needed to compute $A_i \times A_{i+1} \times \cdots \times A_j$ for all $j - i + 1 \leq \ell$.

Proof by Strong Induction on chain length $\ell = j - i + 1$:

Base Case: $\ell = 1$ (i.e., $i = j$)
$dp[i][i] = 0$ is correct. Multiplying a single matrix requires no scalar multiplications

Inductive Hypothesis (IH):
Assume $dp[i][j]$ is correct for all chains of length $< \ell$.

Inductive Step:
Show $dp[i][j]$ is correct for chains of length ℓ.

Consider chain $A_i \times A_{i+1} \times \cdots \times A_j$ where $j - i + 1 = \ell$.

Any valid parenthesization must split the chain at some position k where $i \leq k < j$:

$$(A_i \cdots A_k) \times (A_{k+1} \cdots A_j)$$

Cost analysis for split at k: 1. **Cost to compute left part:** $A_i \cdots A_k$
By **IH**, this is $dp[i][k]$ (chain length $< \ell$), which is correct.

2. **Cost to compute right part:** $A_{k+1} \cdots A_j$
   By **IH**, this is $dp[k+1][j]$ (chain length $< \ell$), which is correct.

3. **Cost to multiply the two results:**

   - Left result has dimensions $p_{i-1} \times p_k$
   - Right result has dimensions $p_k \times p_j$
   - Multiplication cost: $p_{i-1} \cdot p_k \cdot p_j$

Total cost for split at k:

$$\text{Cost}_k = dp[i][k] + dp[k+1][j] + p_{i-1} \cdot p_k \cdot p_j$$

Optimal substructure:
The optimal parenthesization of $A_i \cdots A_j$ must split at some k^*. Moreover: - The optimal cost for the left part $A_i \cdots A_{k^*}$ must be minimal (otherwise we could improve the total by using a better left parenthesization) - Similarly for the right part $A_{k^*+1} \cdots A_j$

Our algorithm tries all possible split points:

$$dp[i][j] = \min_{i \leq k < j} \{\text{Cost}_k\}$$

Since we try all k including k^*, and all subproblem costs are correct by IH:

$$dp[i][j] = \text{Cost}_{k^*} = \text{minimum cost to multiply } A_i \cdots A_j$$

This is correct

Conclusion:
By strong induction on chain length, $dp[i][j]$ is correct for all chain lengths. In particular, $dp[1][n]$ correctly gives the minimum number of scalar multiplications needed for the entire chain. □

---

4. Edit Distance

Problem Statement

Given two strings s_1 and s_2, find the minimum number of operations needed to transform s_1 into s_2.

Allowed operations: 1. **Insert** a character 2. **Delete** a character 3. **Substitute** a character

Also known as **Levenshtein distance**.

Example 1:

```
s1 = "horse"
s2 = "ros"
Output: 3

Explanation:
horse → rorse (substitute 'h' with 'r')
rorse → rose  (delete 'r')
rose  → ros   (delete 'e')
```

Example 2:

```
s1 = "intention"
s2 = "execution"
Output: 5

Explanation:
intention → inention (delete 't')
inention → enention (substitute 'i' with 'e')
enention → exention (substitute 'n' with 'x')
exention → exection (substitute 'n' with 'c')
exection → execution (insert 'u')
```

Worked Example: Step-by-Step

Given: $s_1 = $ `"HORSE"`, $s_2 = $ `"ROS"`

Goal: Transform s_1 to s_2 with minimum operations.

Strategy: Build DP table where $dp[i][j]$ = edit distance between first i characters of s_1 and first j characters of s_2.

Step 1: Initialize Base Cases

Empty strings: - $dp[0][0] = 0$ (both empty, no operations) - $dp[i][0] = i$ (delete all i characters from s_1) - $dp[0][j] = j$ (insert all j characters to get s_2)

Initial Table: | | | R | O | S | |——-|—|—|—|—| | | 0 | 1 | 2 | 3 | | **H** | 1 | | | | | **O** | 2 | | | | | **R** | 3 | | | | | **S** | 4 | | | | | **E** | 5 | | | |

Step 2: Fill Row 1 (H)

dp[1][1]: Transform "H" to "R" - $s_1[1] = $ 'H', $s_2[1] = $ 'R', they differ - Options: 1. Replace 'H' with 'R': $1 + dp[0][0] = 1 + 0 = 1 \leftarrow$ **Best** 2. Delete 'H': $1 + dp[0][1] = 1 + 1 = 2$ 3. Insert 'R' before 'H': $1 + dp[1][0] = 1 + 1 = 2$ - $dp[1][1] = 1$

dp[1][2]: Transform "H" to "RO" - Compare 'H' vs 'O': different - Options: 1. Replace 'H' with 'O': $1+dp[0][1] = 1+1 = 2 \leftarrow$ **Best** 2. Delete 'H': $1+dp[0][2] = 1+2 = 3$ 3. Insert 'O': $1+dp[1][1] = 1+1 = 2$ $\leftarrow$ **Also best** - $dp[1][2] = 2$

dp[1][3]: Transform "H" to "ROS" - Compare 'H' vs 'S': different - Options: 1. Replace: $1 + dp[0][2] = 1 + 2 = 3 \leftarrow$ **Best** 2. Delete: $1 + dp[0][3] = 1 + 3 = 4$ 3. Insert: $1 + dp[1][2] = 1 + 2 = 3 \leftarrow$ **Also best** - $dp[1][3] = 3$

After Row 1: | | | R | O | S | |——-|—|—|—|—| | | 0 | 1 | 2 | 3 | | **H** | 1 | 1 | 2 | 3 | | **O** | 2 | | | | | **R** | 3 | | | | | **S** | 4 | | | | | **E** | 5 | | | |

Step 3: Fill Row 2 (HO)

dp[2][1]: Transform "HO" to "R" - Compare 'O' vs 'R': different - Options: 1. Replace 'O' with 'R': $1 + dp[1][0] = 1 + 1 = 2 \leftarrow$ **Best** 2. Delete 'O': $1 + dp[1][1] = 1 + 1 = 2 \leftarrow$ **Also best** 3. Insert 'R': $1 + dp[2][0] = 1 + 2 = 3$ - $dp[2][1] = 2$

dp[2][2]: Transform "HO" to "RO" - Compare 'O' vs 'O': **same!** - No operation needed: $dp[2][2] = dp[1][1] = 1$

dp[2][3]: Transform "HO" to "ROS" - Compare 'O' vs 'S': different - Options: 1. Replace: $1 + dp[1][2] = 1 + 2 = 3$ 2. Delete: $1 + dp[1][3] = 1 + 3 = 4$ 3. Insert: $1 + dp[2][2] = 1 + 1 = 2 \leftarrow$ **Best** - $dp[2][3] = 2$

After Row 2: | | | R | O | S | |——-|—|—|—|—| | | 0 | 1 | 2 | 3 | | **H** | 1 | 1 | 2 | 3 | | **O** | 2 | 2 | 1 | 2 | | **R** | 3 | | | | | **S** | 4 | | | | | **E** | 5 | | | |

Step 4: Fill Row 3 (HOR)

dp[3][1]: Transform "HOR" to "R" - Compare 'R' vs 'R': **same!** - $dp[3][1] = dp[2][0] = 2$

dp[3][2]: Transform "HOR" to "RO" - Compare 'R' vs 'O': different - Options: 1. Replace: $1+dp[2][1] = 1 + 2 = 3$ 2. Delete: $1 + dp[2][2] = 1 + 1 = 2 \leftarrow$ **Best** 3. Insert: $1 + dp[3][1] = 1 + 2 = 3$ - $dp[3][2] = 2$

dp[3][3]: Transform "HOR" to "ROS" - Compare 'R' vs 'S': different - Options: 1. Replace: $1+dp[2][2] = 1 + 1 = 2 \leftarrow$ **Best** 2. Delete: $1 + dp[2][3] = 1 + 2 = 3$ 3. Insert: $1 + dp[3][2] = 1 + 2 = 3$ - $dp[3][3] = 2$

After Row 3: | | | R | O | S | |——-|—|—|—|—| | | 0 | 1 | 2 | 3 | | **H** | 1 | 1 | 2 | 3 | | **O** | 2 | 2 | 1 | 2 | | **R** | 3 | 2 | 2 | 2 | | **S** | 4 | | | | | **E** | 5 | | | |

Step 5: Fill Row 4 (HORS)

dp[4][1]: "HORS" → "R" - Compare 'S' vs 'R': different - Best: Delete 'S': $1 + dp[3][1] = 1 + 2 = 3$ - $dp[4][1] = 3$

dp[4][2]: "HORS" → "RO" - Compare 'S' vs 'O': different - Best: Delete 'S': $1 + dp[3][2] = 1 + 2 = 3$ - $dp[4][2] = 3$

dp[4][3]: "HORS" → "ROS" - Compare 'S' vs 'S': **same!** - $dp[4][3] = dp[3][2] = 2$

After Row 4: | | | R | O | S | |——-|—|—|—|—| | | 0 | 1 | 2 | 3 | | **H** | 1 | 1 | 2 | 3 | | **O** | 2 | 2 | 1 | 2 | | **R** | 3 | 2 | 2 | 2 | | **S** | 4 | 3 | 3 | 2 | | **E** | 5 | | | |

Step 6: Fill Row 5 (HORSE)

dp[5][1]: "HORSE" → "R" - Compare 'E' vs 'R': different - Best: Delete 'E': $1 + dp[4][1] = 1 + 3 = 4$ - $dp[5][1] = 4$

dp[5][2]: "HORSE" → "RO" - Compare 'E' vs 'O': different - Best: Delete 'E': $1 + dp[4][2] = 1 + 3 = 4$ - $dp[5][2] = 4$

dp[5][3]: "HORSE" → "ROS" - Compare 'E' vs 'S': different - Options: 1. Replace: $1+dp[4][2] = 1+3 = 4$ 2. Delete: $1 + dp[4][3] = 1 + 2 = 3$ ← **Best** 3. Insert: $1 + dp[5][2] = 1 + 4 = 5$ - $dp[5][3] = 3$

Final DP Table: | | | R | O | S | |——-|—|—|—|—| | | 0 | 1 | 2 | 3 | | **H** | 1 | 1 | 2 | 3 | | **O** | 2 | 2 | 1 | 2 | | **R** | 3 | 2 | 2 | 2 | | **S** | 4 | 3 | 3 | 2 | | **E** | 5 | 4 | 4 | **3** |

Answer: $dp[5][3] = 3$ operations needed

One possible sequence: 1. Replace 'H' with 'R': HORSE → RORSE 2. Delete 'R': RORSE → ROSE 3. Delete 'E': ROSE → ROS

Recurrence Relation

State Definition:
Let $dp[i][j]$ = minimum edit distance between $s_1[1..i]$ and $s_2[1..j]$

Recurrence:

$$dp[i][j] = \begin{cases} i & \text{if } j = 0 \quad \text{(delete all } i \text{ chars)} \\ j & \text{if } i = 0 \quad \text{(insert all } j \text{ chars)} \\ dp[i-1][j-1] & \text{if } s_1[i] = s_2[j] \quad \text{(match, no operation)} \\ 1 + \min \begin{cases} dp[i-1][j] & \text{(delete } s_1[i]) \\ dp[i][j-1] & \text{(insert } s_2[j]) \\ dp[i-1][j-1] & \text{(substitute } s_1[i] \to s_2[j]) \end{cases} & \text{otherwise} \end{cases}$$

Final Answer: $dp[m][n]$ where $m = |s_1|$, $n = |s_2|$

Algorithm

```python
def minDistance(s1, s2):
    """
    Returns minimum edit distance between s1 and s2.
    """
    m, n = len(s1), len(s2)

    # Create DP table (size (m+1) × (n+1))
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Base cases
    for i in range(m + 1):
        dp[i][0] = i  # delete all characters
    for j in range(n + 1):
        dp[0][j] = j  # insert all characters

    # Fill table
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s1[i-1] == s2[j-1]:
                # Characters match, no operation needed
                dp[i][j] = dp[i-1][j-1]
            else:
                # Take minimum of three operations
                dp[i][j] = 1 + min(
                    dp[i-1][j],      # delete
                    dp[i][j-1],      # insert
                    dp[i-1][j-1]     # substitute
                )

    return dp[m][n]

# With operation tracking
def minDistanceWithPath(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    ops = [[None] * (n + 1) for _ in range(m + 1)]

    # Base cases
    for i in range(m + 1):
        dp[i][0] = i
        if i > 0:
            ops[i][0] = "DELETE"
    for j in range(n + 1):
        dp[0][j] = j
        if j > 0:
            ops[0][j] = "INSERT"

    # Fill table
```

```python
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = dp[i-1][j-1]
                ops[i][j] = "MATCH"
            else:
                delete_cost = dp[i-1][j]
                insert_cost = dp[i][j-1]
                subst_cost = dp[i-1][j-1]

                min_cost = min(delete_cost, insert_cost, subst_cost)
                dp[i][j] = 1 + min_cost

                if min_cost == delete_cost:
                    ops[i][j] = "DELETE"
                elif min_cost == insert_cost:
                    ops[i][j] = "INSERT"
                else:
                    ops[i][j] = "SUBSTITUTE"

    # Reconstruct operations
    operations = []
    i, j = m, n
    while i > 0 or j > 0:
        op = ops[i][j]
        if op == "MATCH":
            i -= 1
            j -= 1
        elif op == "DELETE":
            operations.append(f"Delete {s1[i-1]} at pos {i}")
            i -= 1
        elif op == "INSERT":
            operations.append(f"Insert {s2[j-1]} at pos {i+1}")
            j -= 1
        elif op == "SUBSTITUTE":
            operations.append(f"Substitute {s1[i-1]} → {s2[j-1]} at pos {i}")
            i -= 1
            j -= 1

    operations.reverse()
    return dp[m][n], operations
```

Time Complexity Analysis

Intuitive Analysis: - **Outer loop:** Iterate i from 1 to $m \to m$ iterations - **Inner loop:** For each i, iterate j from 1 to $n \to n$ iterations - **Each cell:** Constant time $O(1)$ (compare characters, take min of 3 values) - **Total:** $O(m \cdot n)$

Space Complexity: $O(m \cdot n)$ for 2D table

(Can be optimized to $O(\min(m, n))$ using rolling arrays)

Formal Recurrence:

Let $T(m, n)$ = time to compute edit distance for strings of lengths m and n.

Analysis:

```
for i = 1 to m:              // m iterations
    for j = 1 to n:          // n iterations
        O(1) operations      // constant work per cell
```

Recurrence:

$$T(m, n) = \sum_{i=1}^{m} \sum_{j=1}^{n} O(1) = m \cdot n \cdot O(1) = O(mn)$$

Verification: - Total cells to fill: $m \times n$ - Work per cell: - Character comparison: $O(1)$ - Min of 3 values: $O(1)$ - **Total:** $O(mn)$

Space-time tradeoff: - Using full 2D table: Space $O(mn)$, allows path reconstruction - Using two rows: Space $O(n)$, still computes distance but harder to reconstruct path - Using divide-and-conquer (Hirschberg): Space $O(m + n)$, can reconstruct path

---

Correctness Proof

Claim: After computing $dp[i][j]$, it contains the minimum edit distance between $s_1[1..i]$ and $s_2[1..j]$.

Proof by Strong Induction on (i, j) lexicographically:

Base Cases:

Case 1: $i = 0, j = 0$
$dp[0][0] = 0$ is correct. Both strings are empty, no operations needed

Case 2: $i > 0, j = 0$
$dp[i][0] = i$ is correct. Need to delete all i characters from s_1 to get empty string

Case 3: $i = 0, j > 0$
$dp[0][j] = j$ is correct. Need to insert all j characters from s_2 starting with empty string

---

Inductive Hypothesis (IH):
Assume $dp[k][\ell]$ is correct for all (k, ℓ) that are lexicographically smaller than (i, j).
That is: $k < i$, or $k = i$ and $\ell < j$.

Inductive Step:
Show $dp[i][j]$ is correct when we compute it.

When we compute $dp[i][j]$ where $i, j > 0$:

Case A: $s_1[i] = s_2[j]$ **(characters match)**

Algorithm computes: $dp[i][j] = dp[i-1][j-1]$

Justification:
If last characters match, we don't need any operation for them. The edit distance equals the edit distance for the prefixes without those characters.

By **IH**, $dp[i-1][j-1]$ is correct (since $(i-1, j-1)$ is lexicographically smaller than (i, j)).

Therefore, $dp[i][j] = dp[i-1][j-1]$ is correct

Case B: $s_1[i] \neq s_2[j]$ **(characters differ)**

Algorithm computes: $dp[i][j] = 1 + \min\{dp[i-1][j], dp[i][j-1], dp[i-1][j-1]\}$

Any transformation of $s_1[1..i]$ to $s_2[1..j]$ must end with one of three operations:

Operation 1: Delete $s_1[i]$
Transform $s_1[1..i-1]$ to $s_2[1..j]$, then delete $s_1[i]$.
Cost: $1 + dp[i-1][j]$

By **IH**, $dp[i-1][j]$ is correct.

Operation 2: Insert $s_2[j]$
Transform $s_1[1..i]$ to $s_2[1..j-1]$, then insert $s_2[j]$ at the end.
Cost: $1 + dp[i][j-1]$

By **IH**, $dp[i][j-1]$ is correct.

Operation 3: Substitute $s_1[i] \to s_2[j]$
Transform $s_1[1..i-1]$ to $s_2[1..j-1]$, then substitute $s_1[i]$ with $s_2[j]$.
Cost: $1 + dp[i-1][j-1]$

By **IH**, $dp[i-1][j-1]$ is correct.

Optimal substructure:
Any optimal transformation must use one of these three approaches. We cannot avoid dealing with the last character of at least one string.

Moreover, if the optimal transformation uses, say, deletion as the last step, then the transformation of $s_1[1..i-1]$ to $s_2[1..j]$ must also be optimal. If it weren't, we could substitute a better transformation for that part and reduce the total cost, contradicting the assumption of optimality.

Since we take the **minimum** over all three valid options, and all three subproblem costs are correct by IH:

$$dp[i][j] = 1 + \min\{dp[i-1][j], dp[i][j-1], dp[i-1][j-1]\}$$

correctly gives the minimum edit distance

Conclusion:
By strong induction on (i, j), $dp[i][j]$ is correct for all positions. In particular, $dp[m][n]$ correctly gives the minimum edit distance between s_1 and s_2. $\square$

---

5. Longest Palindromic Subsequence

Problem Statement

Given a sequence $S = \langle s_1, s_2, \dots, s_n \rangle$, find the length of the longest palindromic subsequence (LPS).

A **subsequence** is derived by deleting zero or more elements without changing the order of remaining elements.

A **palindrome** reads the same forwards and backwards.

Example 1:

```
S = [1, 2, 3, 4, 2, 1]
LPS = [1, 2, 3, 2, 1] (length 5)
```

Example 2:

```
S = "bbbab"
LPS = "bbbb" (length 4)
```

Example 3:

```
S = "cbbd"
LPS = "bb" (length 2)
```

Worked Example: Step-by-Step

Given: $S = $ `"RACECAR"`

Goal: Find the length of LPS.

Strategy: Use interval DP. Process by increasing substring length.

DP State: $dp[i][j]$ = length of LPS in substring $S[i..j]$

Step 1: Base Case (Length = 1)

Single characters are palindromes of length 1.

```
dp[0][0] = 1  // 'R'
dp[1][1] = 1  // 'A'
dp[2][2] = 1  // 'C'
dp[3][3] = 1  // 'E'
dp[4][4] = 1  // 'C'
dp[5][5] = 1  // 'A'
dp[6][6] = 1  // 'R'
```

Step 2: Length = 2

dp[0][1]: "RA" - $S[0] = $ 'R', $S[1] = $ 'A', differ - $dp[0][1] = \max(dp[1][1], dp[0][0]) = \max(1, 1) = 1$

dp[1][2]: "AC" - 'A' 'C' - $dp[1][2] = \max(dp[2][2], dp[1][1]) = \max(1, 1) = 1$

dp[2][3]: "CE" - 'C' 'E' - $dp[2][3] = 1$

dp[3][4]: "EC" - 'E' 'C' - $dp[3][4] = 1$

dp[4][5]: "CA" - 'C' 'A' - $dp[4][5] = 1$

dp[5][6]: "AR" - 'A' 'R' - $dp[5][6] = 1$

Step 3: Length = 3

dp[0][2]: "RAC" - \$S[0] = \$ 'R', \$S[2] = \$ 'C', differ - $dp[0][2] = \max(dp[0][1], dp[1][2]) = \max(1, 1) = 1$

dp[1][3]: "ACE" - 'A' 'E' - $dp[1][3] = \max(dp[1][2], dp[2][3]) = 1$

dp[2][4]: "CEC" - \$S[2] = \$ 'C', \$S[4] = \$ 'C', **match!** - $dp[2][4] = 2 + dp[3][3] = 2 + 1 = 3$

dp[3][5]: "ECA" - 'E' 'A' - $dp[3][5] = \max(dp[3][4], dp[4][5]) = 1$

dp[4][6]: "CAR" - 'C' 'R' - $dp[4][6] = \max(dp[4][5], dp[5][6]) = 1$

Step 4: Length = 4

dp[0][3]: "RACE" - 'R' 'E' - $dp[0][3] = \max(dp[0][2], dp[1][3]) = \max(1, 1) = 1$

dp[1][4]: "ACEC" - \$S[1] = \$ 'A', \$S[4] = \$ 'C', differ - $dp[1][4] = \max(dp[1][3], dp[2][4]) = \max(1, 3) = 3$

dp[2][5]: "CECA" - 'C' 'A' - $dp[2][5] = \max(dp[2][4], dp[3][5]) = \max(3, 1) = 3$

dp[3][6]: "ECAR" - 'E' 'R' - $dp[3][6] = \max(dp[3][5], dp[4][6]) = \max(1, 1) = 1$

Step 5: Length = 5

dp[0][4]: "RACEC" - 'R' 'C' - $dp[0][4] = \max(dp[0][3], dp[1][4]) = \max(1, 3) = 3$

dp[1][5]: "ACECA" - \$S[1] = \$ 'A', \$S[5] = \$ 'A', **match!** - $dp[1][5] = 2 + dp[2][4] = 2 + 3 = 5$

dp[2][6]: "CECAR" - 'C' 'R' - $dp[2][6] = \max(dp[2][5], dp[3][6]) = \max(3, 1) = 3$

Step 6: Length = 6

dp[0][5]: "RACECA" - 'R' 'A' - $dp[0][5] = \max(dp[0][4], dp[1][5]) = \max(3, 5) = 5$

dp[1][6]: "ACECAR" - 'A' 'R' - $dp[1][6] = \max(dp[1][5], dp[2][6]) = \max(5, 3) = 5$

Step 7: Length = 7

dp[0][6]: "RACECAR" - \$S[0] = \$ 'R', \$S[6] = \$ 'R', **match!** - $dp[0][6] = 2 + dp[1][5] = 2 + 5 = 7$

Final Answer: $dp[0][6] = 7$

The entire string "RACECAR" is itself a palindrome!

LPS: "RACECAR" (all characters included)

Recurrence Relation

State Definition:
Let $dp[i][j]$ = length of longest palindromic subsequence in $S[i..j]$

Recurrence:

$$dp[i][j] = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i > j \\ 2 + dp[i+1][j-1] & \text{if } S[i] = S[j] \\ \max(dp[i+1][j], dp[i][j-1]) & \text{if } S[i] \neq S[j] \end{cases}$$

Explanation: - **Base case:** Single character is a palindrome of length 1 - **Match:** If endpoints match, include both and add 2 to the LPS of the inner substring - **No match:** Try excluding either endpoint and take maximum

Final Answer: $dp[0][n-1]$ for string of length n

Alternative formulation (as LCS):
LPS of S = LCS of S and reverse(S)

Algorithm

```python
def longestPalindromeSubseq(s):
    """
    Returns length of longest palindromic subsequence.
    """
    n = len(s)

    # Create DP table
    dp = [[0] * n for _ in range(n)]

    # Base case: single characters
    for i in range(n):
        dp[i][i] = 1

    # Fill by increasing length
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1

            if s[i] == s[j]:
                if length == 2:
                    dp[i][j] = 2
                else:
                    dp[i][j] = 2 + dp[i+1][j-1]
            else:
                dp[i][j] = max(dp[i+1][j], dp[i][j-1])

    return dp[0][n-1]
```

```python
# With subsequence reconstruction
def longestPalindromeSubseqWithPath(s):
    n = len(s)
    dp = [[0] * n for _ in range(n)]

    for i in range(n):
        dp[i][i] = 1

    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1

            if s[i] == s[j]:
                dp[i][j] = 2 + (dp[i+1][j-1] if length > 2 else 0)
            else:
                dp[i][j] = max(dp[i+1][j], dp[i][j-1])

    # Reconstruct palindrome
    def reconstruct(i, j):
        if i > j:
            return ""
        if i == j:
            return s[i]

        if s[i] == s[j]:
            return s[i] + reconstruct(i+1, j-1) + s[j]
        else:
            if dp[i+1][j] > dp[i][j-1]:
                return reconstruct(i+1, j)
            else:
                return reconstruct(i, j-1)

    palindrome = reconstruct(0, n-1)
    return dp[0][n-1], palindrome

# Alternative: Using LCS with reverse
def longestPalindromeSubseq_LCS(s):
    s_rev = s[::-1]
    return longestCommonSubsequence(s, s_rev)
```

Time Complexity Analysis

Intuitive Analysis: - **Outer loop (length):** Runs from 2 to $n \to O(n)$ iterations - **Inner loop (starting position):** For each length, runs $O(n)$ times - **Each cell:** Constant time $O(1)$ (character comparison, max of 2 values) - Total cells $= \frac{n(n+1)}{2} = O(n^2)$ - **Total Time:** $O(n^2)$

Space Complexity: $O(n^2)$ for the 2D table

Formal Recurrence:

Let $T(n)$ = time to compute LPS for string of length n.

Analysis:

```
for length = 2 to n:              // n-1 iterations
    for i = 0 to n-length:        //   n iterations
        O(1) operations           // constant per cell
```

Recurrence:

$$T(n) = \sum_{\ell=1}^{n} \sum_{i=0}^{n-\ell} O(1)$$

Number of cells to fill:

$$\sum_{\ell=1}^{n} (n - \ell + 1) = \sum_{k=1}^{n} k = \frac{n(n+1)}{2}$$

Therefore:

$$T(n) = O\left(\frac{n(n+1)}{2}\right) = O(n^2)$$

Verification by cell count: - Upper triangular matrix (including diagonal) has $\frac{n(n+1)}{2}$ cells - Each cell takes $O(1)$ time - **Total:** $O(n^2)$

Correctness Proof

Claim: After processing all substrings of length ℓ, $dp[i][j]$ contains the length of the longest palindromic subsequence in $S[i..j]$ for all $j - i + 1 \leq \ell$.

Proof by Strong Induction on substring length $\ell = j - i + 1$:

Base Case: $\ell = 1$ (single characters)
$dp[i][i] = 1$ is correct. A single character is a palindrome of length 1

Inductive Hypothesis (IH):
Assume $dp[i][j]$ is correct for all substrings of length $< \ell$.

Inductive Step:
Show $dp[i][j]$ is correct for substrings of length ℓ where $\ell \geq 2$.

Consider substring $S[i..j]$ where $j - i + 1 = \ell$.

Case 1: $S[i] = S[j]$ **(endpoints match)**

Algorithm computes: $dp[i][j] = 2 + dp[i+1][j-1]$

Justification:
If endpoints match, we can include both in our palindrome. The structure is:

$$\text{LPS of } S[i..j] = S[i] + \text{LPS of } S[i+1..j-1] + S[j]$$

The inner substring $S[i+1..j-1]$ has length $\ell - 2 < \ell$.
By **IH**, $dp[i+1][j-1]$ correctly gives the LPS length of $S[i+1..j-1]$.

Since we're adding matching characters at both ends, the new palindrome has length:

$$2 + dp[i+1][j-1]$$

This is correct

Optimal substructure: If the optimal palindromic subsequence includes both $S[i]$ and $S[j]$ (which it must if they match and are included), then the part between them must also be an optimal palindrome. If not, we could substitute a better inner palindrome and get a better total, contradicting optimality.

Case 2: $S[i] \neq S[j]$ **(endpoints differ)**

Algorithm computes: $dp[i][j] = \max(dp[i+1][j], dp[i][j-1])$

Justification:
If endpoints differ, they cannot both be in the same palindrome. The optimal palindromic subsequence must either: - **Exclude** $S[i]$**:** LPS is in $S[i+1..j]$, length $= dp[i+1][j]$ - **Exclude** $S[j]$**:** LPS is in $S[i..j-1]$, length $= dp[i][j-1]$

Both substrings have length $< \ell$.
By **IH**, both $dp[i+1][j]$ and $dp[i][j-1]$ are correct.

Taking the maximum gives:

$$dp[i][j] = \max(dp[i+1][j], dp[i][j-1])$$

This correctly identifies the longer of the two possibilities

Why not include both endpoints?
If $S[i] \neq S[j]$, including both would create a subsequence that is not a palindrome (different characters at the ends). Therefore, at least one must be excluded.

Optimal substructure: If the optimal solution excludes $S[i]$, then the remaining part $S[i+1..j]$ must have an optimal palindrome. Similar reasoning for excluding $S[j]$.

Conclusion:
By strong induction on substring length, $dp[i][j]$ is correct for all substrings. In particular, $dp[0][n-1]$ correctly gives the length of the longest palindromic subsequence in the entire string. □

---

6. Shortest Common Supersequence

Problem Statement

Given two sequences $x = \langle x_1, x_2, \dots, x_m \rangle$ and $y = \langle y_1, y_2, \dots, y_n \rangle$, find the **shortest common supersequence** (SCS) — the shortest sequence that contains both x and y as subsequences.

Example 1:

```
x = [1, 2, 2, 3]
y = [2, 3, 2]
SCS = [1, 2, 2, 3, 2] or [1, 2, 3, 2, 2]
Output: 5
```

Example 2:

```
x = "AGGTAB"
y = "GXTXAYB"
SCS = "AGGXTXAYB"
Output: 9
```

Key Insight:

$$\text{SCS Length} = |x| + |y| - \text{LCS}(x, y)$$

Where LCS is the Longest Common Subsequence.

Worked Example: Step-by-Step

Given: $x = [1, 2, 2, 3]$, $y = [2, 3, 2]$

Strategy: 1. Find LCS of x and y 2. Use formula: SCS length = $m + n - $ LCS length

---

Step 1: Build LCS Table

$lcs[i][j]$ = length of LCS of $x[0..i-1]$ and $y[0..j-1]$

Initialize: | | | 2 | 3 | 2 | |—|—|—|—|—| | | 0 | 0 | 0 | 0 | | **1** | 0 | | | | | **2** | 0 | | | | | **2** | 0 | | | | | **3** | 0 | | | |

Fill Row 1 (x[0] = 1): - $lcs[1][1]$: Compare 1 vs 2 → differ → $\max(lcs[0][1], lcs[1][0]) = 0$ - $lcs[1][2]$: Compare 1 vs 3 → differ → $\max(lcs[0][2], lcs[1][1]) = 0$ - $lcs[1][3]$: Compare 1 vs 2 → differ → $\max(lcs[0][3], lcs[1][2]) = 0$

		2	3	2
	0	0	0	0
1	0	0	0	0
2	0			
2	0			
3	0			

Fill Row 2 (x[1] = 2): - $lcs[2][1]$: Compare 2 vs 2 → **match!** → $lcs[1][0] + 1 = 0 + 1 = 1$ - $lcs[2][2]$: Compare 2 vs 3 → differ → $\max(lcs[1][2], lcs[2][1]) = \max(0, 1) = 1$ - $lcs[2][3]$: Compare 2 vs 2 → **match!** → $lcs[1][2] + 1 = 0 + 1 = 1$

		2	3	2
	0	0	0	0
1	0	0	0	0
2	0	1	1	1
2	0			
3	0			

Fill Row 3 (x[2] = 2): - $lcs[3][1]$: Compare 2 vs 2 → **match!** → $lcs[2][0] + 1 = 0 + 1 = 1$ - $lcs[3][2]$: Compare 2 vs 3 → differ → $\max(lcs[2][2], lcs[3][1]) = \max(1, 1) = 1$ - $lcs[3][3]$: Compare 2 vs 2 → **match!** → $lcs[2][2] + 1 = 1 + 1 = 2$

		2	3	2
	0	0	0	0
1	0	0	0	0
2	0	1	1	1
2	0	1	1	2
3	0			

Fill Row 4 (x[3] = 3): - $lcs[4][1]$: Compare 3 vs 2 → differ → $\max(lcs[3][1], lcs[4][0]) = \max(1, 0) = 1$ - $lcs[4][2]$: Compare 3 vs 3 → **match!** → $lcs[3][1] + 1 = 1 + 1 = 2$ - $lcs[4][3]$: Compare 3 vs 2 → differ → $\max(lcs[3][3], lcs[4][2]) = \max(2, 2) = 2$

Final LCS Table: | | | 2 | 3 | 2 | |—|—|—|—|—| | | 0 | 0 | 0 | 0 | | **1** | 0 | 0 | 0 | 0 | | **2** | 0 | 1 | 1 | 1 | | **2** | 0 | 1 | 1 | 2 | | **3** | 0 | 1 | **2** | 2 |

LCS Length: $lcs[4][3] = 2$

One possible LCS: $[2, 3]$ (or $[2, 2]$)

Step 2: Compute SCS Length

Using formula:

$$\text{SCS length} = |x| + |y| - \text{LCS length} = 4 + 3 - 2 = 5$$

Step 3: Construct Actual SCS (Optional)

Backtrack through LCS table to build the supersequence: - When characters match: include once - When they differ: include both

One possible SCS: $[1, 2, 2, 3, 2]$ - 1 from x only - 2 appears in both (from LCS) - Another 2 from x - 3 appears in both (from LCS) - Final 2 from y

Answer: Length = 5

Recurrence Relation

Since SCS is computed via LCS, we need the LCS recurrence first.

LCS State Definition:
Let $lcs[i][j]$ = length of longest common subsequence of $x[1..i]$ and $y[1..j]$

LCS Recurrence:

$$lcs[i][j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ lcs[i-1][j-1] + 1 & \text{if } x[i] = y[j] \\ \max(lcs[i-1][j], lcs[i][j-1]) & \text{otherwise} \end{cases}$$

SCS Formula:

$$\text{SCS}(x, y) = |x| + |y| - lcs[m][n]$$

where $m = |x|$, $n = |y|$

Why this works: - Total elements needed: $|x| + |y|$ - Subtract common elements (LCS): they're counted twice but should appear once - Result: minimum supersequence length

---

Algorithm

```python
def shortestCommonSupersequence(x, y):
    """
    Returns length of shortest common supersequence.
    """
    m, n = len(x), len(y)

    # Step 1: Compute LCS table
    lcs = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if x[i-1] == y[j-1]:
                lcs[i][j] = lcs[i-1][j-1] + 1
            else:
                lcs[i][j] = max(lcs[i-1][j], lcs[i][j-1])

    # Step 2: Apply SCS formula
    return m + n - lcs[m][n]

# With actual supersequence construction
def shortestCommonSupersequenceString(x, y):
    m, n = len(x), len(y)

    # Build LCS table
    lcs = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if x[i-1] == y[j-1]:
                lcs[i][j] = lcs[i-1][j-1] + 1
            else:
                lcs[i][j] = max(lcs[i-1][j], lcs[i][j-1])

    # Reconstruct SCS by backtracking
    scs = []
    i, j = m, n

    while i > 0 and j > 0:
        if x[i-1] == y[j-1]:
            # Common element: include once
            scs.append(x[i-1])
            i -= 1
            j -= 1
        elif lcs[i-1][j] > lcs[i][j-1]:
            # Element from x
            scs.append(x[i-1])
```

```
            i -= 1
        else:
            # Element from y
            scs.append(y[j-1])
            j -= 1

    # Add remaining elements
    while i > 0:
        scs.append(x[i-1])
        i -= 1
    while j > 0:
        scs.append(y[j-1])
        j -= 1

    scs.reverse()
    return scs
```

Time Complexity Analysis

Intuitive Analysis: - **LCS computation:** $O(mn)$ to fill the table - **SCS formula application:** $O(1)$ arithmetic - **Total:** $O(mn)$

Space: $O(mn)$ for LCS table (can be optimized to $O(\min(m, n))$)

Formal Recurrence:

Let $T(m, n)$ = time to compute SCS for sequences of lengths m and n.

Analysis:

$$T(m, n) = T_{\text{LCS}}(m, n) + O(1)$$

LCS Time Complexity:

```
for i = 1 to m:              // m iterations
    for j = 1 to n:          // n iterations
        O(1) operations      // constant per cell
```

$$T_{\text{LCS}}(m, n) = \sum_{i=1}^{m} \sum_{j=1}^{n} O(1) = m \cdot n \cdot O(1) = O(mn)$$

SCS computation: After LCS table is built, compute:

$$\text{SCS length} = m + n - lcs[m][n]$$

This is $O(1)$ arithmetic.

Total:

$$T(m, n) = O(mn) + O(1) = O(mn)$$

Verification: - LCS table has $(m+1) \times (n+1)$ cells - Each cell computed in $O(1)$ time - Formula evaluation is $O(1)$ - **Total:** $O(mn)$

Correctness Proof

Claim: The formula $\text{SCS}(x, y) = |x| + |y| - \text{LCS}(x, y)$ correctly computes the length of the shortest common supersequence.

Proof Strategy: We prove both existence and optimality.

Part 1: Existence (Upper Bound)

We show that a supersequence of length $m + n - L$ exists, where $L = |\text{LCS}(x, y)|$.

Construction:

Let $z = \langle z_1, z_2, \dots, z_L \rangle$ be an LCS of x and y.

Build supersequence S as follows: 1. Process x and y simultaneously 2. When both sequences have the next LCS element, include it once 3. Include all non-LCS elements from both sequences

Algorithm:

```
S = []
i = 0, j = 0, k = 0  // pointers for x, y, LCS

while k < L:
    // Add elements from x until next LCS element
    while x[i] != z[k]:
        S.append(x[i])
        i += 1

    // Add elements from y until next LCS element
    while y[j] != z[k]:
        S.append(y[j])
        j += 1

    // Add the common LCS element once
    S.append(z[k])
    i += 1
    j += 1
    k += 1

// Add remaining elements
S.extend(x[i:])
S.extend(y[j:])
```

Length of S: - All m elements from x are included - All n elements from y are included - But L common elements (LCS) are counted only once, not twice - **Length:** $m + n - L$

Verification that S is a supersequence: - x is a subsequence of S: All elements of x appear in S in order - y is a subsequence of S: All elements of y appear in S in order

Therefore, a supersequence of length $m + n - L$ exists

Part 2: Optimality (Lower Bound)

Claim: No supersequence can be shorter than $m + n - L$.

Proof by counting:

Let S be any common supersequence of x and y.

Since S is a supersequence: - x is a subsequence of $S \to$ at least m positions in S correspond to elements of x - y is a subsequence of $S \to$ at least n positions in S correspond to elements of y

Key observation: Positions in S can correspond to both x and y if they represent common elements.

Let: - X_{only} = positions in S corresponding only to x (not shared with y) - Y_{only} = positions in S corresponding only to y (not shared with x) - XY_{common} = positions in S corresponding to both x and y (shared)

Then:

$$|S| = |X_{\text{only}}| + |Y_{\text{only}}| + |XY_{\text{common}}|$$

Constraints: - $|X_{\text{only}}| + |XY_{\text{common}}| \geq m$ (must cover all of x) - $|Y_{\text{only}}| + |XY_{\text{common}}| \geq n$ (must cover all of y)

Adding these:

$$|X_{\text{only}}| + |Y_{\text{only}}| + 2|XY_{\text{common}}| \geq m + n$$

Therefore:

$$|S| = |X_{\text{only}}| + |Y_{\text{only}}| + |XY_{\text{common}}| \geq m + n - |XY_{\text{common}}|$$

Maximum sharing:
The maximum number of positions that can be shared is the length of the LCS:

$$|XY_{\text{common}}| \leq L$$

with equality when S is constructed optimally.

Therefore:

$$|S| \geq m + n - L$$

Combined with Part 1 (existence of supersequence with length $m + n - L$):

$$|S_{\text{optimal}}| = m + n - L$$

Conclusion:
The shortest common supersequence has length exactly $m + n - L$, where $L = |\text{LCS}(x, y)|$. Since LCS can be computed correctly (proven separately), the SCS formula is correct. □

---

7. Beautiful Sequence (Minimum Additions for Palindrome)

Problem Statement

Given a sequence $X = \langle x_1, x_2, \dots, x_n \rangle$, find the minimum number of elements to add to make it "beautiful" (a palindrome - reads the same forwards and backwards). Elements can be added anywhere in the sequence.

Examples:

```
X = [1, 2, 3, 4]
Solution: Add [3, 2, 1] → [1, 2, 3, 4, 3, 2, 1]
Minimum additions: 3

X = [1, 2, 3, 2, 1]
Solution: Already palindromic
Minimum additions: 0

X = [1, 2, 3, 2]
Solution: Add [1] → [1, 2, 3, 2, 1]
Minimum additions: 1
```

Worked Example: Step-by-Step

Given: $X = \langle 1, 2, 3, 4 \rangle$

Goal: Find minimum additions to make palindrome.

Key Insight: Find the longest palindromic subsequence (LPS) first!

Step 1: Build LPS Table (same as Problem 5)

	0	**1**	**2**	**3**
0 (1)	1	1	1	1
1 (2)		1	1	1
2 (3)			1	1
3 (4)				1

Filling by increasing length: - Length 1 (base): All diagonal cells = 1 - Length 2: $dp[0][1]$: 1 2 $\rightarrow$ $\max(dp[0][0], dp[1][1]) = 1$ - Length 3: $dp[0][2]$: 1 3 $\rightarrow$ $\max(dp[0][1], dp[1][2]) = 1$ - Length 4: $dp[0][3]$: 1 4 $\rightarrow$ $\max(dp[0][2], dp[1][3]) = 1$

LPS Length = 1 (any single character is palindromic)

Step 2: Apply Formula

$$\text{MinAdditions} = n - \text{LPS}(X) = 4 - 1 = 3$$

Step 3: Construct Beautiful Sequence

One approach: Mirror the non-LPS elements

- LPS: Just "1" (or any single element)
- Non-LPS elements: 2, 3, 4 (need mirrors)
- Construction: Start with X, add reversed elements at end
- Result: $\langle 1, 2, 3, 4, 3, 2, 1 \rangle$

Check: Reads same forwards and backwards? **YES**
Additions: 3 elements (3, 2, 1)

Recurrence Relation

The Beautiful Sequence problem reduces to the Longest Palindromic Subsequence problem:

$$\text{MinAdditions}(X) = n - \text{LPS}(X)$$

Where LPS is computed using:

$$\text{LPS}[i][j] = \begin{cases} 1 & \text{if } i = j \\ 2 & \text{if } i + 1 = j \text{ and } X[i] = X[j] \\ 2 + \text{LPS}[i+1][j-1] & \text{if } X[i] = X[j] \\ \max(\text{LPS}[i+1][j], \text{LPS}[i][j-1]) & \text{if } X[i] \neq X[j] \end{cases}$$

Intuition: - LPS tells us which elements already form a palindromic structure - The remaining $n - \text{LPS}$ elements need mirrored counterparts - Each unpaired element requires one addition

---

Algorithm

```python
def minAdditionsForBeautiful(X):
    """
    Returns minimum number of additions to make X palindromic.
    """
    n = len(X)

    # Step 1: Compute LPS using interval DP
    dp = [[0] * n for _ in range(n)]

    # Base case: single characters
    for i in range(n):
        dp[i][i] = 1

    # Fill by increasing length
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1

            if X[i] == X[j]:
                if length == 2:
                    dp[i][j] = 2
                else:
                    dp[i][j] = 2 + dp[i+1][j-1]
            else:
                dp[i][j] = max(dp[i+1][j], dp[i][j-1])
```

```python
    lps_length = dp[0][n-1]

    # Step 2: Apply formula
    min_additions = n - lps_length

    return min_additions

def constructBeautifulSequence(X):
    """
    Returns one possible beautiful sequence.
    """
    # Method: mirror non-LPS elements
    n = len(X)
    result = list(X)

    # Simple construction: add reverse at the end
    # For proper construction, would need to track LPS path
    for i in range(n - 1, -1, -1):
        result.append(X[i])

    # Note: This creates a palindrome but may not be minimal
    # For minimal construction, need to identify non-LPS elements

    return result
```

Note: The algorithm focuses on computing the minimum number. Constructing the actual beautiful sequence requires tracking which elements are in the LPS during the DP computation (similar to LCS path reconstruction).

Time Complexity Analysis

Intuitive Analysis: - **Step 1:** Compute LPS using interval DP $\rightarrow O(n^2)$ - **Step 2:** Subtract LPS from n $\rightarrow O(1)$ - **Total Time:** $O(n^2)$

Space: $O(n^2)$ for DP table

Formal Recurrence:

Since Beautiful Sequence reduces to LPS:

$$T_{\text{Beautiful}}(n) = T_{\text{LPS}}(n) + O(1)$$

From Problem 5, we know $T_{\text{LPS}}(n) = O(n^2)$

Therefore:

$$T_{\text{Beautiful}}(n) = O(n^2) + O(1) = O(n^2)$$

Detailed breakdown:

LPS computation:

```
for length = 2 to n:              // n-1 iterations
    for i = 0 to n-length:        //   n iterations per length
        j = i + length - 1
        // O(1) work per cell: comparison, max of 2 values
```

Total cells filled: $\sum_{\ell=1}^{n}(n-\ell+1) = \frac{n(n+1)}{2} = O(n^2)$

Each cell: $O(1)$ work

Total: $O(n^2)$

Correctness Proof

Claim: The minimum number of elements to add to sequence X to make it palindromic is exactly $n - \text{LPS}(X)$.

Proof (Two Parts: Achievability + Optimality)

Part 1: Achievability (Upper Bound)

We show that $n - L$ additions are sufficient, where $L = \text{LPS}(X)$.

Construction Algorithm:

Let $P = \langle p_1, p_2, \dots, p_L \rangle$ be a longest palindromic subsequence of X with positions $i_1, i_2, \dots, i_L$ in X.

Build beautiful sequence B:

1. Start with all elements of X
2. For each element $x_k \notin P$ (not in the palindromic subsequence):
    - Add its mirror at the appropriate position to maintain palindrome property

Alternative simple construction: - Append reverse of non-LPS elements: If LPS uses L elements, append the $n - L$ unused elements in reverse

Example: $X = [1, 2, 3, 4]$, LPS $= [1]$ (length 1) - Non-LPS elements: 2, 3, 4 - Append their mirrors: $[1, 2, 3, 4] \to [1, 2, 3, 4, 3, 2, 1]$ - Check: Palindrome? YES - Additions: $3 = n - L$

General verification:

After adding $n - L$ elements: - Total length: $n + (n - L) = 2n - L$ - Structure: Elements from LPS form palindromic core, others are mirrored - Result is palindromic

Therefore, $n - L$ additions are achievable

Part 2: Optimality (Lower Bound)

We show that fewer than $n - L$ additions are insufficient.

Proof by counting argument:

In the final palindrome B: - B has length $n + k$ where $k =$ number of additions - B must be palindromic

Counting positions:

For X to be a subsequence of B: - All n elements of X appear in B - At most L of these elements can be "self-paired" (palindromically positioned) because LPS has length L - The remaining $n - L$ elements must each be paired with an added element (their mirror)

Therefore: - Minimum additions needed: $n - L$ (one mirror for each non-LPS element) - If $k < n - L$: Not enough mirrors added → impossible to form palindrome - Contradiction

Therefore: $k \geq n - L$

Conclusion:

We've shown: - $k = n - L$ is achievable (Part 1) - $k < n - L$ is impossible (Part 2)

Therefore, the minimum number of additions is exactly $n - \text{LPS}(X)$. □

---

8. Subset Sum Problem

Problem Statement

Given a set $S = \{s_1, s_2, \dots, s_n\}$ of nonnegative integers and a target sum t, determine whether there exists a subset of S whose elements sum precisely to t.

Examples:

```
S = {3, 1, 5, 2}, t = 6
Answer: YES (subsets: {1, 5} or {1, 2, 3})

S = {3, 1, 5, 2}, t = 10
Answer: YES (subset: {3, 5, 2})

S = {3, 1, 5, 2}, t = 12
Answer: NO (max sum = 11)

S = {1, 2, 3}, t = 0
Answer: YES (empty subset)
```

Worked Example: Step-by-Step

Given: $S = \{3, 1, 5, 2\}$, $t = 6$

Goal: Determine if subset exists that sums to 6.

Strategy: Build DP table where $dp[i][j]$ = "can we make sum j using first i elements?"

Step 1: Initialize Base Cases

- $dp[i][0]$ = true for all i (empty subset sums to 0)
- $dp[0][j]$ = false for all $j > 0$ (no elements, can't make positive sum)

Step 2: Fill DP Table

Table 6: rows = elements {3, 1, 5, 2}, columns = sums 0 through 6

	0	**1**	**2**	**3**	**4**	**5**	**6**
{ }	T	F	F	F	F	F	F
{3}	T	F	F	T	F	F	F
{3,1}	T	T	F	T	T	F	F
{3,1,5}	T	T	F	T	T	T	T
{3,1,5,2}	T	T	T	T	T	T	T

Filling process:

Row 1: Add element 3 - $dp[1][3]$: Can make 3? Take 3 → YES (from $dp[0][0]$) - $dp[1][4]$: Can make 4? $3 < 4$, check $dp[0][4]$ or $dp[0][1]$ → NO

Row 2: Add element 1 - $dp[2][1]$: Can make 1? Take 1 → YES (from $dp[1][0]$) - $dp[2][4]$: Can make 4? Take 1, need 3 → Check $dp[1][3]$ → YES

Row 3: Add element 5 - $dp[3][5]$: Can make 5? Take 5 → YES (from $dp[2][0]$) - $dp[3][6]$: Can make 6? Take 5, need 1 → Check $dp[2][1]$ → YES

Row 4: Add element 2 - $dp[4][2]$: Can make 2? Take 2 → YES - $dp[4][6]$: Already true from row 3, stays true

Answer: $dp[4][6]$ = **TRUE**

Reconstruction: From $dp[4][6]$: - Row 4 added element 2. Is $dp[3][6]$ also true? YES → Element 2 not used in this path. - Row 3 added element 5. Is $dp[2][1]$ true? YES → Element 5 used! (6 - 5 = 1) - Row 2 added element 1. Is $dp[1][0]$ true? YES → Element 1 used! (1 - 1 = 0)

Subset found: {1, 5} with sum = 6

Recurrence Relation

Let $dp[i][j]$ = true if we can form sum j using the first i elements of S.

$$dp[i][j] = \begin{cases} \text{true} & \text{if } j = 0 \text{ (empty subset)} \\ \text{false} & \text{if } i = 0 \text{ and } j > 0 \text{ (no elements left)} \\ dp[i-1][j] & \text{if } S[i-1] > j \text{ (element too large)} \\ dp[i-1][j] \vee dp[i-1][j - S[i-1]] & \text{otherwise} \end{cases}$$

Interpretation:

- **Base case 1:** Sum 0 is always achievable with empty subset
- **Base case 2:** Without elements, cannot make positive sum
- **Case 1:** If $S[i-1]$ (current element) exceeds target j, cannot include it → check previous row
- **Case 2:** Two options:
  - **Exclude** $S[i-1]$: Check $dp[i-1][j]$
  - **Include** $S[i-1]$: Check if we can make $j - S[i-1]$ with previous elements → $dp[i-1][j-S[i-1]]$
  - **Result:** TRUE if either option works (logical OR)

Algorithm

```python
def subsetSum(S, t):
    """
    Returns True if subset of S sums to t, False otherwise.
    """
    n = len(S)

    # Create DP table
    dp = [[False] * (t + 1) for _ in range(n + 1)]

    # Base case: sum 0 is always possible with empty subset
    for i in range(n + 1):
        dp[i][0] = True

    # Fill table
    for i in range(1, n + 1):
        for j in range(1, t + 1):
            # Option 1: Don't include S[i-1]
            dp[i][j] = dp[i-1][j]

            # Option 2: Include S[i-1] if it fits
            if j >= S[i-1]:
                dp[i][j] = dp[i][j] or dp[i-1][j - S[i-1]]

    return dp[n][t]

def reconstructSubset(S, t):
    """
    Returns one subset that sums to t, or None if no such subset exists.
    """
    n = len(S)
    dp = [[False] * (t + 1) for _ in range(n + 1)]

    # Fill table (same as above)
    for i in range(n + 1):
        dp[i][0] = True

    for i in range(1, n + 1):
        for j in range(1, t + 1):
            dp[i][j] = dp[i-1][j]
            if j >= S[i-1]:
                dp[i][j] = dp[i][j] or dp[i-1][j - S[i-1]]

    if not dp[n][t]:
        return None  # No solution

    # Backtrack to find subset
    subset = []
    i, j = n, t
```

```python
    while i > 0 and j > 0:
        # Was element S[i-1] used?
        if dp[i][j] and not dp[i-1][j]:
            # Yes, it was used
            subset.append(S[i-1])
            j -= S[i-1]
        i -= 1

    return subset

# Space-optimized version (1D array)
def subsetSum_optimized(S, t):
    """
    Space-optimized O(t) version.
    """
    dp = [False] * (t + 1)
    dp[0] = True  # Base case

    for num in S:
        # Traverse backward to avoid using updated values in same iteration
        for j in range(t, num - 1, -1):
            dp[j] = dp[j] or dp[j - num]

    return dp[t]
```

Key implementation detail: In space-optimized version, iterate j **backward** to ensure we use values from the "previous row" (previous iteration), not updated values from current iteration.

Time Complexity Analysis

Intuitive Analysis: - **Fill** $(n+1) \times (t+1)$ **table** - Each cell: $O(1)$ computation (boolean check, logical OR) - **Total cells:** $(n+1) \times (t+1) = O(nt)$ - **Time:** $O(nt)$

Space: - Standard: $O(nt)$ for 2D table - Optimized: $O(t)$ for 1D array

---

Formal Recurrence:

Let $T(n, t)$ = time to solve subset sum for n elements and target t.

```
for i = 1 to n:              // n iterations
    for j = 1 to t:          // t iterations
        // O(1) work per cell
```

$$T(n, t) = \sum_{i=1}^{n} \sum_{j=1}^{t} O(1) = n \times t \times O(1) = O(nt)$$

Note on pseudo-polynomial time:

This is **pseudo-polynomial** because t is a numeric value, not input size.

- If $t = 2^k$ (exponential in bit representation), then $O(nt) = O(n \cdot 2^k)$ becomes exponential in true input size.
- Subset Sum is NP-complete for general inputs.
- DP solution is efficient when t is reasonably small.

Correctness Proof

Claim: After processing i elements, $dp[i][j]$ is true if and only if there exists a subset of $\{S[0], S[1], \dots, S[i-1]\}$ with sum exactly j.

Proof by Strong Induction on (i, j) (lexicographic order)

Base Cases:

1. $dp[0][0] =$ **true**
   - Empty subset has sum 0 $\rightarrow$ Correct
2. $dp[0][j] =$ **false for** $j > 0$
   - No elements available, cannot make positive sum $\rightarrow$ Correct
3. $dp[i][0] =$ **true for all** $i \geq 0$
   - Empty subset always sums to 0 $\rightarrow$ Correct

Inductive Hypothesis:

Assume $dp[i'][j']$ is correct for all (i', j') that come before (i, j) in lexicographic order (i.e., $i' < i$ or ($i' = i$ and $j' < j$)).

Inductive Step:

Show $dp[i][j]$ is correct for $i \geq 1, j \geq 1$.

When computing $dp[i][j]$, consider element $S[i-1]$:

Case 1: $S[i-1] > j$ **(element exceeds target)**

```
dp[i][j] = dp[i-1][j]
```

Analysis: - Element $S[i-1]$ cannot be included (too large) - Therefore, $dp[i][j]$ depends only on whether subsets of $\{S[0], \dots, S[i-2]\}$ can sum to j - By **IH**, $dp[i-1][j]$ correctly indicates this (since $(i-1, j)$ comes before (i, j))

Correctness: $dp[i][j]$ correctly represents "can first i elements make sum j"

Case 2: $S[i-1] \leq j$ **(element can potentially be included)**

```
dp[i][j] = dp[i-1][j] or dp[i-1][j - S[i-1]]
```

Analysis:

Two mutually exclusive options for making sum j with first i elements:

Option A: Exclude $S[i-1]$ - Subsets of $\{S[0], \dots, S[i-2]\}$ sum to j - By **IH**, $dp[i-1][j]$ correctly indicates this

Option B: Include $S[i-1]$ - Subsets of $\{S[0], \dots, S[i-2]\}$ sum to $j - S[i-1]$, then add $S[i-1]$ to reach j - By **IH**, $dp[i-1][j-S[i-1]]$ correctly indicates this (since $(i-1, j-S[i-1])$ comes before (i,j))

Logical OR correctness:

If $dp[i][j]$ **should be true:** - There exists a subset of first i elements summing to j - This subset either: - Excludes $S[i-1] \to dp[i-1][j] = \text{true} \to dp[i][j] = \text{true}$ - Includes $S[i-1] \to$ Remaining elements sum to $j - S[i-1] \to dp[i-1][j-S[i-1]] = \text{true} \to dp[i][j] = \text{true}$

If $dp[i][j]$ **should be false:** - No subset of first i elements sums to j - This means: - Subsets of first $i-1$ elements don't sum to $j \to dp[i-1][j] = \text{false}$ - Subsets of first $i-1$ elements don't sum to $j - S[i-1] \to dp[i-1][j-S[i-1]] = \text{false}$ - Therefore: $dp[i][j] = \text{false} \vee \text{false} = \text{false}$

Correctness: $dp[i][j]$ correctly represents "can first i elements make sum j"

Conclusion:

By strong induction on (i,j) in lexicographic order, $dp[i][j]$ is correct for all positions.

In particular, $dp[n][t]$ correctly determines whether a subset of S sums to target t. $\square$

Advanced Problems and Classic DP (9-26)

9. 0/1 Knapsack Problem

Problem Statement

Given n items where item i has weight w_i and value v_i, and a knapsack with capacity W, select items to maximize total value without exceeding capacity. Each item can be taken at most once.

Example: Items (w,v): $(2,3), (3,4), (4,5), (5,6)$, Capacity $W = 8 \to$ Take items 2,4: value $= 10$

Worked Example

Items: $w = [2,3,4,5]$, $v = [3,4,5,6]$, $W = 8$

Key decision per item: Take it or leave it?

DP Table (rows=items, cols=capacity):

Item	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1(2,3)	0	0	3	3	3	3	3	3	3
2(3,4)	0	0	3	4	4	7	7	7	7
3(4,5)	0	0	3	4	5	7	8	9	9
4(5,6)	0	0	3	4	5	7	8	9	10

Answer: $dp[4][8] = 10$ (items 2,4)

Recurrence Relation

$$dp[i][w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ dp[i-1][w] & \text{if } w_i > w \\ \max(dp[i-1][w], dp[i-1][w-w_i] + v_i) & \text{otherwise} \end{cases}$$

Algorithm

```
def knapsack(weights, values, W):
    n = len(weights)
    dp = [[0]*(W+1) for _ in range(n+1)]

    for i in range(1, n+1):
        for w in range(1, W+1):
            dp[i][w] = dp[i-1][w]  # Don't take
            if weights[i-1] <= w:
                dp[i][w] = max(dp[i][w],
                               dp[i-1][w-weights[i-1]] + values[i-1])
    return dp[n][W]
```

Time Complexity

Time: $O(nW)$ — Fill $(n+1) \times (W+1)$ table, $O(1)$ per cell
Space: $O(nW)$ standard, $O(W)$ optimized (iterate w backwards)

Correctness Proof

Claim: $dp[i][w]$ = optimal value using first i items with capacity w.

Proof by induction on (i, w):

Base: $dp[0][w] = 0$ (no items), $dp[i][0] = 0$ (no capacity)

IH: Assume correct for all (i', w') before (i, w).

Step: For $dp[i][w]$, optimal solution either: - Excludes item $i \to$ value $= dp[i-1][w]$ (by IH) - Includes item $i \to$ value $= v_i + dp[i-1][w-w_i]$ (by IH)

Algorithm takes max, so $dp[i][w]$ is optimal. □

---

10. Longest Increasing Subsequence (LIS)

Problem Statement

Find length of longest subsequence where elements are in strictly increasing order.

Example: $[10, 9, 2, 5, 3, 7, 101, 18] \to$ LIS: $[2, 3, 7, 101]$, length $= 4$

Worked Example

Array: $[10, 9, 2, 5, 3, 7, 101, 18]$

$O(n^2)$ **DP approach:** $dp[i]$ = length of LIS ending at index i

i	arr[i]	dp[i]	Explanation
0	10	1	Single element
1	9	1	$9 < 10$, can't extend
2	2	1	$2 < 9$, can't extend
3	5	2	$2 < 5$, extend dp[2]
4	3	2	$2 < 3$, extend dp[2]
5	7	3	$5 < 7$, extend dp[3]
6	101	4	$7 < 101$, extend dp[5]
7	18	4	$7 < 18$, extend dp[5]

Answer: $\max(dp) = 4$

Recurrence Relations

$O(n^2)$ **version:**

$$dp[i] = \max\{dp[j] + 1 \mid j < i \text{ and } arr[j] < arr[i]\}$$

Base: $dp[i] = 1$ (single element)

$O(n \log n)$ **version:**
Use binary search with "tails" array where $tails[len]$ = smallest tail element of all LIS of length $len + 1$.

Algorithms

```python
# O(n^2) solution
def lengthOfLIS_n2(nums):
    n = len(nums)
    dp = [1] * n

    for i in range(1, n):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)

    return max(dp)

# O(n log n) solution
import bisect
def lengthOfLIS_nlogn(nums):
    tails = []
```

```python
    for num in nums:
        pos = bisect.bisect_left(tails, num)
        if pos == len(tails):
            tails.append(num)
        else:
            tails[pos] = num
    return len(tails)
```

Time Complexity

$O(n^2)$ **version:** Two nested loops $\rightarrow O(n^2)$
$O(n \log n)$ **version:** n elements, $O(\log n)$ binary search each $\rightarrow O(n \log n)$

Space: $O(n)$ for both versions

Correctness Proof ($O(n^2)$ version)

Claim: $dp[i]$ = length of LIS ending at index i.

Proof by induction on i:

Base: $dp[0] = 1$ (single element)

IH: Assume $dp[k]$ correct for all $k < i$.

Step: LIS ending at i either: - Starts at i: length $= 1$ - Extends LIS from some $j < i$ where $arr[j] < arr[i]$: length $= dp[j] + 1$

By IH, all $dp[j]$ are correct. Algorithm takes max over all valid j, so $dp[i]$ is correct. $\square$

---

11. Rod Cutting Problem

Problem Statement

Given rod of length n and prices $p[1..n]$, determine maximum revenue by cutting rod and selling pieces.

Example: $n = 4$, prices $[1, 5, 8, 9] \rightarrow$ Cut as $2 + 2$: revenue $= 10$

Worked Example

Length: $n = 4$
Prices: $p[1] = 1, p[2] = 5, p[3] = 8, p[4] = 9$

DP Table:

Length	0	1	2	3	4
Revenue	0	1	5	8	10

Computation: - $r[1] = 1$ (no cut) - $r[2] = \max(p[2], p[1] + r[1]) = \max(5, 2) = 5$ - $r[3] = \max(p[3], p[1] + r[2], p[2] + r[1]) = \max(8, 6, 6) = 8$ - $r[4] = \max(p[4], p[1] + r[3], p[2] + r[2], p[3] + r[1]) = \max(9, 9, 10, 9) = 10$

Optimal: Cut into lengths 2+2

Recurrence Relation

$$r[n] = \max_{i=1}^{n} \{p[i] + r[n-i]\}$$

Interpretation: Try all possible first cuts of length i, recursively solve remainder.

Algorithm

```python
def rodCutting(prices, n):
    r = [0] * (n+1)

    for length in range(1, n+1):
        max_rev = 0
        for i in range(1, length+1):
            max_rev = max(max_rev, prices[i] + r[length-i])
        r[length] = max_rev

    return r[n]
```

Time Complexity

Time: $O(n^2)$ — for each length, try all cut positions
Space: $O(n)$

$$T(n) = \sum_{i=1}^{n} O(i) = O(1 + 2 + ... + n) = O(n^2)$$

Correctness Proof

Claim: $r[n]$ = max revenue for rod of length n.

Proof by strong induction on n:

Base: $r[0] = 0$ (no rod, no revenue)

IH: Assume $r[k]$ correct for all $k < n$.

Step: Optimal solution makes some first cut of length i, leaving rod of length $n - i$. By IH, $r[n-i]$ is optimal. Total revenue $= p[i] + r[n-i]$. Algorithm tries all i, takes max, so $r[n]$ is correct. □

---

12. Maximum Subarray Sum (Kadane's Algorithm)

Problem Statement

Find contiguous subarray with maximum sum.

Example: $[-2, 1, -3, 4, -1, 2, 1, -5, 4] \rightarrow [4, -1, 2, 1]$, sum $= 6$

Worked Example

Array: $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$

Iteration:

i	arr[i]	curr_sum	best_sum
0	-2	-2	-2
1	1	1 (reset)	1
2	-3	-2	1
3	4	4 (reset)	4
4	-1	3	4
5	2	5	5
6	1	6	6
7	-5	1	6
8	4	5	6

Answer: 6 (subarray from indices 3-6)

Recurrence Relation

$$\text{maxEndingHere}[i] = \max\{arr[i], \text{maxEndingHere}[i-1] + arr[i]\}$$

Answer: $\max_i\{\text{maxEndingHere}[i]\}$

Algorithm

```python
def maxSubArray(nums):
    max_ending_here = nums[0]
    max_so_far = nums[0]

    for i in range(1, len(nums)):
        max_ending_here = max(nums[i], max_ending_here + nums[i])
        max_so_far = max(max_so_far, max_ending_here)

    return max_so_far
```

Time Complexity

Time: $O(n)$ — single pass
Space: $O(1)$ — two variables

Correctness Proof

Loop invariant: At iteration i, maxEndingHere = max sum of subarray ending at i.

Proof:

Init: $i = 0$, maxEndingHere = $nums[0]$

Maintenance: At i, subarray ending at i either: - Starts at i: sum = $nums[i]$ - Extends from $i-1$: sum = maxEndingHere$[i-1] + nums[i]$

Algorithm computes max of these

Termination: Loop covers all positions. maxSoFar tracks maximum across all endings. □

---

13. Climbing Stairs

Problem Statement

Climb stairs with n steps. Can climb 1 or 2 steps at a time. How many distinct ways to reach top?

Example: $n = 3 \rightarrow 3$ ways: $(1+1+1), (1+2), (2+1)$

Worked Example

Stairs: $n = 5$

n	Ways	Computation
1	1	(1)
2	2	(1+1), (2)
3	3	ways[1]+ways[2] = 1+2
4	5	ways[2]+ways[3] = 2+3
5	8	ways[3]+ways[4] = 3+5

Answer: 8 ways

Pattern: Fibonacci sequence!

Recurrence Relation

$$\text{ways}[n] = \text{ways}[n-1] + \text{ways}[n-2]$$

Base cases: ways[1] = 1, ways[2] = 2

Algorithm

```python
def climbStairs(n):
    if n <= 2:
        return n

    prev2, prev1 = 1, 2
    for i in range(3, n+1):
        current = prev1 + prev2
        prev2, prev1 = prev1, current

    return prev1
```

Time Complexity

Time: $O(n)$ — single loop
Space: $O(1)$ — optimized to two variables

Correctness Proof

Claim: $\text{ways}[n]$ = number of distinct ways to climb n stairs.

Proof by strong induction:

Base: $\text{ways}[1] = 1$, $\text{ways}[2] = 2$

IH: Assume correct for all $k < n$.

Step: To reach stair n, must come from either: - Stair $n-1$ (take 1 step): $\text{ways}[n-1]$ ways (by IH) - Stair $n-2$ (take 2 steps): $\text{ways}[n-2]$ ways (by IH)

These are mutually exclusive and exhaustive, so $\text{ways}[n] = \text{ways}[n-1] + \text{ways}[n-2]$. $\square$

---

14. House Robber

Problem Statement

Rob houses along street. Adjacent houses have connected security. Maximize money robbed.

Example: $[1, 2, 3, 1] \rightarrow$ Rob houses 0,2: money $= 1 + 3 = 4$

Worked Example

Houses: $[2, 7, 9, 3, 1]$

i	money[i]	dp[i]	Explanation
0	2	2	Rob house 0
1	7	7	Rob house 1 (better than 2)

i	money[i]	dp[i]	Explanation
2	9	11	Rob 0+2: 2+9=11
3	3	11	Max(dp[2], dp[1]+3)=max(11,10)=11
4	1	12	Max(dp[3], dp[2]+1)=max(11,12)=12

Answer: 12 (rob houses 0,2,4)

Recurrence Relation

$$dp[i] = \max\{dp[i-1], dp[i-2] + \text{money}[i]\}$$

Base: $dp[0] = \text{money}[0]$, $dp[1] = \max(\text{money}[0], \text{money}[1])$

Algorithm

```python
def rob(nums):
    if not nums:
        return 0
    if len(nums) == 1:
        return nums[0]

    prev2 = nums[0]
    prev1 = max(nums[0], nums[1])

    for i in range(2, len(nums)):
        current = max(prev1, prev2 + nums[i])
        prev2, prev1 = prev1, current

    return prev1
```

Time Complexity

Time: $O(n)$
Space: $O(1)$ optimized

Correctness Proof

Claim: $dp[i]$ = max money robbing houses 0 to i.

Proof by induction on i:

Base: $dp[0]$ and $dp[1]$ correct by definition

IH: Assume $dp[k]$ correct for all $k < i$.

Step: Optimal solution either: - Skips house i: value $= dp[i-1]$ (by IH) - Robs house i: value $=$ money$[i] + dp[i-2]$ (can't rob $i-1$, by IH)

Algorithm takes max, so $dp[i]$ is correct. □

---

15. Coin Change (Minimum Coins)

Problem Statement

Given coins of different denominations and amount, find minimum coins needed.

Example: coins $= [1, 2, 5]$, amount $= 11 \rightarrow 3$ coins: $(5 + 5 + 1)$

Worked Example

Coins: $[1, 2, 5]$, **Amount:** 11

DP Table:

Amount	0	1	2	3	4	5	6	7	8	9	10	11
Min coins	0	1	1	2	2	1	2	2	3	3	2	3

Computation: - $dp[6] = \min(dp[5] + 1, dp[4] + 1, dp[1] + 1) = \min(2, 3, 2) = 2$ (use coin 5 or coin 1) - $dp[11] = \min(dp[10] + 1, dp[9] + 1, dp[6] + 1) = \min(3, 4, 3) = 3$

Answer: 3 coins

Recurrence Relation

$$dp[a] = \min_{c \in \text{coins}, c \leq a} \{dp[a - c] + 1\}$$

Base: $dp[0] = 0$

Algorithm

```python
def coinChange(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0

    for a in range(1, amount + 1):
        for coin in coins:
            if coin <= a:
                dp[a] = min(dp[a], dp[a - coin] + 1)

    return dp[amount] if dp[amount] != float('inf') else -1
```

Time Complexity

Time: $O(n \cdot amount)$ where $n = |\text{coins}|$
Space: $O(amount)$

Correctness Proof

Claim: $dp[a]$ = minimum coins to make amount a.

Proof by strong induction on a:

Base: $dp[0] = 0$

IH: Assume $dp[k]$ correct for all $k < a$.

Step: To make amount a, must use some coin c. By IH, $dp[a-c]$ is optimal. Total coins $= 1 + dp[a-c]$. Algorithm tries all coins, takes min, so $dp[a]$ is correct. $\square$

---

16. Word Break

Problem Statement

Given string s and dictionary, determine if s can be segmented into dictionary words.

Example: s = "leetcode", dict = ["leet", "code"] $\rightarrow$ True

---

Worked Example

String: s = "applepenapple"
Dictionary: ["apple", "pen"]

DP array: $dp[i]$ = can segment $s[0..i-1]$?

i	substring	dp[i]	Explanation
0	""	T	Empty string
5	"apple"	T	In dict
8	"applepen"	T	"apple"+"pen"
13	"applepenapple"	T	"applepen"+"apple"

Answer: True

Recurrence Relation

$$dp[i] = \bigvee_{j=0}^{i-1} (dp[j] \wedge s[j..i-1] \in \text{dict})$$

Base: $dp[0]$ = true (empty string)

Algorithm

```python
def wordBreak(s, wordDict):
    n = len(s)
    dp = [False] * (n + 1)
    dp[0] = True
    word_set = set(wordDict)

    for i in range(1, n + 1):
        for j in range(i):
            if dp[j] and s[j:i] in word_set:
                dp[i] = True
                break

    return dp[n]
```

Time Complexity

Time: $O(n^2 \cdot m)$ where m = avg word length (substring check)
Space: $O(n)$

Correctness Proof

Claim: $dp[i]$ = True iff $s[0..i-1]$ can be segmented.

Proof by induction on i:

Base: $dp[0]$ = True (empty string)

IH: Assume $dp[k]$ correct for all $k < i$.

Step: $s[0..i-1]$ can be segmented iff exists split point j where: - $s[0..j-1]$ can be segmented (dp[j] by IH) - $s[j..i-1]$ is in dictionary

Algorithm checks all j, so $dp[i]$ is correct. □

---

17. Partition Problem

Problem Statement

Can set be partitioned into two subsets with equal sum?

Example: $[1, 5, 11, 5] \rightarrow$ Yes: $\{1, 5, 5\}$ and $\{11\}$

Worked Example

Set: $[1, 5, 11, 5]$
Total sum: 22, **Target:** 11

This reduces to **Subset Sum** with target = 11!

DP Table: (subset sum for target 11)

	0	1	...	11
{ }	T	F	...	F
{1}	T	T	...	F
{1,5}	T	T	...	F
{1,5,11}	T	T	...	T

Answer: True (can partition)

Recurrence Relation

If total sum is odd → False

Otherwise, reduce to subset sum:

$$dp[i][s] = dp[i-1][s] \vee dp[i-1][s - nums[i-1]]$$

Target: sum/2

Algorithm

```python
def canPartition(nums):
    total = sum(nums)
    if total % 2 != 0:
        return False

    target = total // 2
    dp = [False] * (target + 1)
    dp[0] = True

    for num in nums:
        for s in range(target, num - 1, -1):
            dp[s] = dp[s] or dp[s - num]

    return dp[target]
```

Time Complexity

Time: $O(n \cdot \text{target})$ where target = sum/2
Space: $O(\text{target})$

Correctness Proof

Follows directly from Subset Sum correctness (Problem 8). Partition exists iff subset with sum = total/2 exists. □

---

18. Egg Drop Problem

Problem Statement

With k eggs and n floors, find minimum attempts needed in worst case to find critical floor.

Example: $k = 2, n = 10 \rightarrow 4$ attempts

Worked Example

Eggs: $k = 2$, **Floors:** $n = 10$

Strategy: Try floor x. If breaks, search below with $k - 1$ eggs. If survives, search above with k eggs.

DP Table (simplified):

k\n	1	2	3	4	5
1	1	2	3	4	5
2	1	2	2	3	3

For $k = 2, n = 5$: Try floor 3 first. Worst case $= 1 + \max(dp[1][2], dp[2][2]) = 1 + 2 = 3$

Recurrence Relation

$$dp[k][n] = 1 + \min_{x=1}^{n} \max\{dp[k-1][x-1], dp[k][n-x]\}$$

Base cases: - $dp[k][0] = 0$, $dp[k][1] = 1$ - $dp[1][n] = n$ (linear search with 1 egg)

Algorithm

```python
def eggDrop(k, n):
    dp = [[0] * (n + 1) for _ in range(k + 1)]

    for i in range(1, k + 1):
        dp[i][1] = 1
    for j in range(1, n + 1):
        dp[1][j] = j

    for eggs in range(2, k + 1):
        for floors in range(2, n + 1):
            dp[eggs][floors] = float('inf')
```

```
            for x in range(1, floors + 1):
                worst = 1 + max(dp[eggs-1][x-1], dp[eggs][floors-x])
                dp[eggs][floors] = min(dp[eggs][floors], worst)

    return dp[k][n]
```

Time Complexity

Time: $O(kn^2)$ — for each (k, n), try n floors
Space: $O(kn)$

Correctness Proof

Claim: $dp[k][n]$ = min attempts with k eggs and n floors.

Proof by strong induction on (k, n):

Base: $dp[1][n] = n$, $dp[k][1] = 1$

IH: Assume correct for all smaller (k', n').

Step: When we drop from floor x: - Breaks: $dp[k-1][x-1]$ attempts remain (by IH) - Survives: $dp[k][n-x]$ attempts remain (by IH) - Worst case: max of these - Optimal: minimize over all x

Algorithm implements this, so $dp[k][n]$ is correct. □

---

19. Longest Common Substring

Problem Statement

Find the length of the longest contiguous substring that appears in both strings X and Y.

Key difference from LCS: Must be **contiguous** (no gaps allowed).

Example: X = "ABABC", Y = "BABCA" → Longest common substring: "BABC", length = 4

Worked Example

String X: "ABABC" (length 5)
String Y: "BABCA" (length 5)

DP Table: $dp[i][j]$ = length of common substring **ending at** $X[i-1]$ and $Y[j-1]$

		B	**A**	**B**	**C**	**A**
	0	0	0	0	0	0
A	0	0	**1**	0	0	**1**
B	0	**1**	0	**2**	0	0
A	0	0	**2**	0	0	**1**
B	0	**1**	0	**3**	0	0

		B	**A**	**B**	**C**	**A**
C	0	0	0	0	**4**	0

Computation examples: - $dp[2][3] = 2$: "AB" from $X[1:2]$ matches $Y[0:1]$ - $dp[4][3] = 3$: "BAB" from $X[1:3]$ matches $Y[0:2]$ - $dp[5][4] = 4$: **"BABC"** from $X[1:4]$ matches $Y[0:3] \leftarrow$ **Maximum!**

Answer: $\max_{i,j} dp[i][j] = 4$

Recurrence Relation

$$dp[i][j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ dp[i-1][j-1] + 1 & \text{if } X[i-1] = Y[j-1] \\ 0 & \text{if } X[i-1] \neq Y[j-1] \end{cases}$$

Critical insight: Reset to 0 on mismatch (enforces contiguity).

Algorithm

```python
def longestCommonSubstring(X, Y):
    m, n = len(X), len(Y)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    max_length = 0
    end_pos_x = 0

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if X[i-1] == Y[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
                if dp[i][j] > max_length:
                    max_length = dp[i][j]
                    end_pos_x = i
            else:
                dp[i][j] = 0

    # Substring: X[end_pos_x - max_length : end_pos_x]
    return max_length
```

Time Complexity

Time: $O(mn)$ — Fill $(m+1) \times (n+1)$ table, $O(1)$ per cell
Space: $O(mn)$ standard, $O(n)$ optimized (only need previous row)

Correctness Proof

Claim: $dp[i][j]$ = length of longest common substring ending at $X[i-1]$ and $Y[j-1]$.

Proof by strong induction on (i, j):

Base: $dp[0][j] = 0$, $dp[i][0] = 0$ (no string, no substring)

IH: Assume correct for all (i', j') with $i' < i$ or $j' < j$.

Step: For $dp[i][j]$, consider two cases:

1. **If $X[i-1] \neq Y[j-1]$:**
   No common substring ending at these positions $\rightarrow dp[i][j] = 0$
2. **If $X[i-1] = Y[j-1]$:**
   Common substring ending here extends from positions $(i-1, j-1)$.
   By IH, $dp[i-1][j-1]$ is correct.
   Length $= dp[i-1][j-1] + 1$

Algorithm tracks max over all positions, finding longest substring anywhere. □

---

20. Palindrome Partitioning (Minimum Cuts)

Problem Statement

Given string s, partition it into minimum number of cuts such that every substring is a palindrome.

Example: s = "aab" $\rightarrow$ 1 cut: "aa|b" (both "aa" and "b" are palindromes)

Worked Example

String: s = "aabba" (length 5)

Step 1: Build palindrome table ($isPalin[i][j]$ = is $s[i : j+1]$ a palindrome?)

	0	1	2	3	4
0(a)	T	T	F	F	T
1(a)		T	F	T	T
2(b)			T	T	F
3(b)				T	F
4(a)					T

Step 2: Compute minimum cuts ($cuts[i]$ = min cuts for $s[0 : i]$)

i	s[0:i]	cuts[i]	Computation
0	""	0	Base case
1	"a"	0	Palindrome, no cuts
2	"aa"	0	Palindrome, no cuts
3	"aab"	1	"aa|b", $cuts[2] + 1 = 1$
4	"aabb"	1	"aabb" not palin, but "aa|bb", $cuts[2] + 1 = 1$

i	s[0:i]	cuts[i]	Computation
5	"aabba"	0	Entire string is palindrome!

Answer: 0 cuts (entire string "aabba" is a palindrome)

Recurrence Relation

Palindrome table:

$$isPalin[i][j] = \begin{cases} \text{True} & \text{if } i \geq j \\ s[i] = s[j] \wedge (j - i \leq 2 \text{ or } isPalin[i+1][j-1]) & \text{otherwise} \end{cases}$$

Cuts:

$$cuts[i] = \begin{cases} 0 & \text{if } isPalin[0][i-1] \\ \min_{j=1}^{i-1}\{cuts[j] + 1 \mid isPalin[j][i-1]\} & \text{otherwise} \end{cases}$$

Algorithm

```python
def minCut(s):
    n = len(s)

    # Build palindrome table
    isPalin = [[False] * n for _ in range(n)]
    for i in range(n):
        isPalin[i][i] = True

    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            if s[i] == s[j]:
                isPalin[i][j] = (length == 2) or isPalin[i+1][j-1]

    # Compute minimum cuts
    cuts = [0] * (n + 1)
    for i in range(1, n + 1):
        if isPalin[0][i-1]:
            cuts[i] = 0
        else:
            cuts[i] = float('inf')
            for j in range(1, i):
                if isPalin[j][i-1]:
                    cuts[i] = min(cuts[i], cuts[j] + 1)

    return cuts[n]
```

Time Complexity

Time: $O(n^2)$
- Build palindrome table: $O(n^2)$ (all substrings) - Compute cuts: $O(n^2)$ (for each position, try all split points)

Space: $O(n^2)$ for palindrome table + $O(n)$ for cuts array

Correctness Proof

Claim: $cuts[i]$ = minimum cuts to partition $s[0:i]$.

Proof by strong induction on i:

Base: $cuts[0] = 0$ (empty string)

IH: Assume $cuts[k]$ correct for all $k < i$.

Step: For $s[0:i]$, consider two cases:

1. **If $s[0:i]$ is itself a palindrome:**
   No cuts needed $\rightarrow cuts[i] = 0$
2. **Otherwise, must cut somewhere:**
   Try all split points j where $s[j:i]$ is palindrome.
   By IH, $cuts[j]$ is optimal for prefix.
   Total cuts $= cuts[j] + 1$ (one cut at position j).
   Algorithm tries all j, takes min, so $cuts[i]$ is correct.

□

---

21. Optimal Binary Search Tree

Problem Statement

Given n keys with search frequencies $freq[1..n]$, construct BST that minimizes expected search cost.

Cost formula: cost $= \sum_{i=1}^{n} freq[i] \times depth[i]$

Example: Keys $[10, 12, 20]$, freq $[34, 8, 50] \rightarrow$ Root at 20 minimizes cost

---

Worked Example

Keys: $[10, 12, 20]$
Frequencies: $[34, 8, 50]$

Try all possible root choices:

1. **Root = 10:**

```
10
  \
  12
    \
```

```
      20
```

Cost $= 34 \times 1 + 8 \times 2 + 50 \times 3 = 34 + 16 + 150 = 200$

2. **Root = 12:**

```
     12
    /  \
  10    20
```

Cost $= 34 \times 2 + 8 \times 1 + 50 \times 2 = 68 + 8 + 100 = 176$

3. **Root = 20:**

```
     20
     /
    12
    /
  10
```

Cost $= 34 \times 3 + 8 \times 2 + 50 \times 1 = 102 + 16 + 50 = 168 \leftarrow$ **Optimal!**

Answer: Minimum cost = 168

Recurrence Relation

Let $dp[i][j]$ = min cost for keys i to j

$$dp[i][j] = \begin{cases} 0 & \text{if } i > j \\ \min_{r=i}^{j} \left\{ dp[i][r-1] + dp[r+1][j] + \sum_{k=i}^{j} freq[k] \right\} & \text{otherwise} \end{cases}$$

Interpretation: - Choose root r: cost to access root - Left subtree: $dp[i][r-1]$ - Right subtree: $dp[r+1][j]$ - Depth increase: $\sum freq[k]$ (all nodes go one level deeper)

Algorithm

```python
def optimalBST(keys, freq):
    n = len(keys)
    dp = [[0] * n for _ in range(n)]

    # Precompute frequency sums
    freq_sum = [[0] * n for _ in range(n)]
    for i in range(n):
        freq_sum[i][i] = freq[i]
        for j in range(i + 1, n):
            freq_sum[i][j] = freq_sum[i][j-1] + freq[j]

    # Fill DP table for increasing interval lengths
    for length in range(1, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
```

```
            dp[i][j] = float('inf')

            for r in range(i, j + 1):
                left_cost = dp[i][r-1] if r > i else 0
                right_cost = dp[r+1][j] if r < j else 0
                cost = left_cost + right_cost + freq_sum[i][j]
                dp[i][j] = min(dp[i][j], cost)

    return dp[0][n-1]
```

Time Complexity

Time: $O(n^3)$ - $O(n^2)$ intervals - For each interval, try $O(n)$ roots - Frequency sum: $O(1)$ with precomputation

Space: $O(n^2)$ for DP table + frequency sums

Correctness Proof

Claim: $dp[i][j]$ = minimum cost BST for keys i to j.

Proof by strong induction on interval size $j - i + 1$:

Base: $dp[i][i] = freq[i]$ (single key, cost = frequency)
$dp[i][i-1] = 0$ (empty range)

IH: Assume correct for all intervals smaller than $[i, j]$.

Step: For interval $[i, j]$, optimal tree has some root r. This divides into: - Left subtree: keys $[i, r-1]$ with cost $dp[i][r-1]$ (by IH) - Right subtree: keys $[r+1, j]$ with cost $dp[r+1][j]$ (by IH) - All nodes descend one level: add $\sum_{k=i}^{j} freq[k]$

Total cost $= dp[i][r-1] + dp[r+1][j] + \sum freq[k]$

Algorithm tries all roots, takes min, so $dp[i][j]$ is correct. □

---

22. Matrix Chain Multiplication

See Problem 3 for complete solution — Optimal parenthesization, $O(n^3)$, interval DP pattern.

---

23. Longest Palindromic Substring

Problem Statement

Find the longest contiguous substring that is a palindrome.

Difference from LPS: LPS finds longest palindromic **subsequence** (can skip chars). This requires **contiguous** palindrome.

Example: s = "babad" → "bab" or "aba", length = 3

Worked Example

String: $s =$ "racecar" (length 7)

DP Table: $dp[i][j] =$ is substring $s[i : j+1]$ a palindrome?

	0	1	2	3	4	5	6
0(r)	T	F	F	F	F	F	T
1(a)		T	F	F	F	T	F
2(c)			T	F	T	F	F
3(e)				T	F	F	F
4(c)					T	F	F
5(a)						T	F
6(r)							T

Key entries: - $dp[0][6] =$ True: "racecar" (length 7) $\leftarrow$ **Maximum!** - $dp[1][5] =$ True: "aceca" (length 5) - $dp[2][4] =$ True: "cec" (length 3) - All diagonals are True (single chars)

Answer: "racecar", length = 7

Recurrence Relation

$$dp[i][j] = \begin{cases} \text{True} & \text{if } i = j \\ s[i] = s[j] & \text{if } j = i+1 \\ s[i] = s[j] \wedge dp[i+1][j-1] & \text{if } j > i+1 \end{cases}$$

Fill order: Increasing substring length (diagonally from bottom-left to top-right)

Algorithm

```python
def longestPalindrome(s):
    n = len(s)
    if n == 0:
        return ""

    dp = [[False] * n for _ in range(n)]
    start = 0
    max_len = 1

    # All single characters are palindromes
    for i in range(n):
        dp[i][i] = True

    # Check for length 2
    for i in range(n - 1):
```

```
        if s[i] == s[i + 1]:
            dp[i][i + 1] = True
            start = i
            max_len = 2

    # Check for lengths 3 to n
    for length in range(3, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1

            if s[i] == s[j] and dp[i + 1][j - 1]:
                dp[i][j] = True
                start = i
                max_len = length

    return s[start : start + max_len]
```

Time Complexity

Time: $O(n^2)$ - $O(n^2)$ substrings to check - Each check is $O(1)$ using memoization

Space: $O(n^2)$ for DP table

Note: Expand-around-center approach can also solve in $O(n^2)$ time with $O(1)$ space.

Correctness Proof

Claim: $dp[i][j] = \text{True}$ iff $s[i : j + 1]$ is a palindrome.

Proof by strong induction on substring length $\ell = j - i + 1$:

Base:
- $\ell = 1$: $dp[i][i] = \text{True}$ (single char is palindrome) - $\ell = 2$: $dp[i][i + 1] = (s[i] = s[i + 1])$ (check both chars)

IH: Assume correct for all lengths $< \ell$.

Step: For length $\ell > 2$, substring $s[i : j + 1]$ is palindrome iff: 1. First and last chars match: $s[i] = s[j]$ 2. Inner substring is palindrome: $dp[i + 1][j - 1]$ (length $\ell - 2$, by IH)

Algorithm checks both conditions, so $dp[i][j]$ is correct. $\square$

---

24. Minimum Path Sum in Grid

Problem Statement

Given $m \times n$ grid with non-negative numbers, find path from top-left to bottom-right that minimizes sum. Can only move right or down.

Example:
Grid:

```
1  3  1
1  5  1
4  2  1
```

Minimum path: $1 \to 3 \to 1 \to 1 \to 1$, sum $= 7$

Worked Example

Grid: (4×3)

```
1  2  3
4  5  6
7  8  9
1  2  3
```

DP Table: $dp[i][j]$ = minimum path sum to reach (i, j)

	0	**1**	**2**
0	**1**	3 (1+2)	6 (3+3)
1	5 (1+4)	**8** (min(5+5,3+5))	12 (8+6)
2	12 (5+7)	16 (8+8)	21 (min(16+9,12+9))
3	13 (12+1)	15 (13+2)	**18** (15+3)

Step-by-step: - Start: $dp[0][0] = 1$ - First row: $dp[0][1] = 1 + 2 = 3$, $dp[0][2] = 3 + 3 = 6$ - First col: $dp[1][0] = 1+4 = 5$, $dp[2][0] = 5+7 = 12$ - Interior: $dp[1][1] = \min(dp[0][1], dp[1][0]) + 5 = \min(3, 5) + 5 = 8$ - Final: $dp[3][2] = 18 \leftarrow$ **Answer**

Path: $(0, 0) \to (0, 1) \to (1, 1) \to (2, 1) \to (3, 1) \to (3, 2)$

Recurrence Relation

$$dp[i][j] = \begin{cases} grid[0][0] & \text{if } i = 0, j = 0 \\ dp[i-1][j] + grid[i][j] & \text{if } j = 0 \\ dp[i][j-1] + grid[i][j] & \text{if } i = 0 \\ \min(dp[i-1][j], dp[i][j-1]) + grid[i][j] & \text{otherwise} \end{cases}$$

Algorithm

```python
def minPathSum(grid):
    m, n = len(grid), len(grid[0])
    dp = [[0] * n for _ in range(m)]

    # Initialize starting position
    dp[0][0] = grid[0][0]

    # Fill first row
    for j in range(1, n):
```

```
        dp[0][j] = dp[0][j-1] + grid[0][j]

    # Fill first column
    for i in range(1, m):
        dp[i][0] = dp[i-1][0] + grid[i][0]

    # Fill rest of table
    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = min(dp[i-1][j], dp[i][j-1]) + grid[i][j]

    return dp[m-1][n-1]
```

Space optimization: Use 1D array (only need previous row):

```
def minPathSum_optimized(grid):
    m, n = len(grid), len(grid[0])
    dp = [float('inf')] * n
    dp[0] = 0

    for i in range(m):
        dp[0] = dp[0] + grid[i][0]  # First column
        for j in range(1, n):
            dp[j] = min(dp[j], dp[j-1]) + grid[i][j]

    return dp[n-1]
```

Time Complexity

Time: $O(mn)$ — Visit each cell once
Space: $O(mn)$ standard, $O(n)$ optimized (single row)

Correctness Proof

Claim: $dp[i][j]$ = minimum path sum from $(0, 0)$ to (i, j).

Proof by strong induction on (i, j) in row-major order:

Base: $dp[0][0] = grid[0][0]$ (starting point)

IH: Assume correct for all (i', j') before (i, j) in row-major order.

Step: To reach (i, j), must come from either: - Above: $(i - 1, j)$ with cost $dp[i - 1][j]$ (by IH) - Left: $(i, j - 1)$ with cost $dp[i][j - 1]$ (by IH)

Minimum path to (i, j) = min of these two + $grid[i][j]$.

Algorithm computes this, so $dp[i][j]$ is correct. □

---

25. Unique Paths in Grid

Problem Statement

Robot located at top-left of $m \times n$ grid. Can only move right or down. Count distinct paths to bottom-right corner.

Example: $m = 3, n = 3 \rightarrow 6$ unique paths

Worked Example

Grid size: $m = 3$ rows, $n = 3$ columns

DP Table: $dp[i][j]$ = number of paths to reach (i, j)

	0	**1**	**2**
0	1	1	1
1	1	2	3
2	1	3	**6**

Computation: - First row: 1 path (all right moves) - First col: 1 path (all down moves) - $dp[1][1] = dp[0][1] + dp[1][0] = 1 + 1 = 2$ - $dp[1][2] = dp[0][2] + dp[1][1] = 1 + 2 = 3$ - $dp[2][1] = dp[1][1] + dp[2][0] = 2 + 1 = 3$ - $dp[2][2] = dp[1][2] + dp[2][1] = 3 + 3 = 6 \leftarrow$ **Answer**

The 6 paths: 1. RRDD 2. RDRD 3. RDDR 4. DRRD 5. DRDR 6. DDRR

(R = right, D = down)

Recurrence Relation

$$dp[i][j] = \begin{cases} 1 & \text{if } i = 0 \text{ or } j = 0 \\ dp[i-1][j] + dp[i][j-1] & \text{otherwise} \end{cases}$$

Closed-form solution:
Need $(m - 1)$ down moves and $(n - 1)$ right moves.
Total paths $= \binom{m+n-2}{m-1} = \binom{m+n-2}{n-1}$

Algorithm

DP approach:

```python
def uniquePaths(m, n):
    dp = [[1] * n for _ in range(m)]

    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = dp[i-1][j] + dp[i][j-1]

    return dp[m-1][n-1]
```

Space-optimized (O(n)):

```python
def uniquePaths_optimized(m, n):
    dp = [1] * n

    for i in range(1, m):
        for j in range(1, n):
            dp[j] = dp[j] + dp[j-1]

    return dp[n-1]
```

Combinatorial approach (O(min(m,n))):

```python
def uniquePaths_math(m, n):
    from math import comb
    return comb(m + n - 2, m - 1)
```

Time Complexity

DP approach: - **Time:** $O(mn)$ — Fill entire table - **Space:** $O(mn)$ standard, $O(n)$ optimized

Combinatorial approach: - **Time:** $O(\min(m, n))$ — Compute binomial coefficient - **Space:** $O(1)$

Correctness Proof

Claim: $dp[i][j]$ = number of distinct paths from $(0, 0)$ to (i, j).

Proof by strong induction on (i, j):

Base:
- $dp[0][j] = 1$ for all j (only one path: all right) - $dp[i][0] = 1$ for all i (only one path: all down)

IH: Assume correct for all (i', j') with $i' \leq i$ and $j' < j$, or $i' < i$.

Step: To reach (i, j), robot must arrive from either: - Above: $(i-1, j) \to dp[i-1][j]$ paths (by IH) - Left: $(i, j-1) \to dp[i][j-1]$ paths (by IH)

These are mutually exclusive (different last moves), so total paths = sum.

Algorithm computes $dp[i][j] = dp[i-1][j] + dp[i][j-1]$, so correct. □

Combinatorial proof:
From $(0, 0)$ to $(m-1, n-1)$ requires $(m-1)$ down + $(n-1)$ right = $(m+n-2)$ total moves.
Choose which $(m-1)$ positions are "down" moves: $\binom{m+n-2}{m-1}$ ways. □

---

26. Interleaving String

Problem Statement

Given three strings s_1, s_2, and s_3, determine if s_3 is formed by interleaving s_1 and s_2.

Interleaving: Divide s_1 and s_2 into substrings, then combine them alternately (preserving original order).

Example:
s_1 = "aabcc", s_2 = "dbbca", s_3 = "aadbbcbcac" → True
Interleaving: **aa** (from s_1) + **dbb** (from s_2) + **c** (from s_1) + **b** (from s_2) + **c** (from s_1) + **a** (from s_2) + **c** (from s_1)

Worked Example

Strings:
s_1 = "abc" (length 3)
s_2 = "def" (length 3)
s_3 = "adbecf" (length 6)

DP Table: $dp[i][j]$ = can $s_3[0:i+j]$ be formed by interleaving $s_1[0:i]$ and $s_2[0:j]$?

		d	**e**	**f**
	T	F	F	F
a	T	T	F	F
b	F	T	T	F
c	F	F	T	T

Key computations: - $dp[0][0]$ = True (empty strings) - $dp[1][0]$ = True: "a" matches $s_3[0]$ - $dp[1][1]$ = True: "ad" can be formed ($s_1[0]$ + $s_2[0]$) - $dp[2][1]$ = True: "adb" = "a" + "d" + "b" - $dp[3][3]$ = True: "adbecf" is valid interleaving

Answer: True

Recurrence Relation

$$dp[i][j] = \begin{cases} \text{True} & \text{if } i = 0 \text{ and } j = 0 \\ (dp[i-1][j] \wedge s_1[i-1] = s_3[i+j-1]) & \text{if } i > 0 \\ \vee & \\ (dp[i][j-1] \wedge s_2[j-1] = s_3[i+j-1]) & \text{if } j > 0 \end{cases}$$

Interpretation:
$s_3[0:i+j]$ can be formed by interleaving if: - Previous $i-1$ chars of s_1 + all j of s_2 work, AND $s_1[i-1]$ matches current char - **OR** all i of s_1 + previous $j-1$ of s_2 work, AND $s_2[j-1]$ matches current char

---

Algorithm

```python
def isInterleave(s1, s2, s3):
    m, n = len(s1), len(s2)

    # Length check
    if m + n != len(s3):
```

```
        return False

    dp = [[False] * (n + 1) for _ in range(m + 1)]
    dp[0][0] = True

    # Fill first column (only s1)
    for i in range(1, m + 1):
        dp[i][0] = dp[i-1][0] and s1[i-1] == s3[i-1]

    # Fill first row (only s2)
    for j in range(1, n + 1):
        dp[0][j] = dp[0][j-1] and s2[j-1] == s3[j-1]

    # Fill rest of table
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            dp[i][j] = (dp[i-1][j] and s1[i-1] == s3[i+j-1]) or \
                       (dp[i][j-1] and s2[j-1] == s3[i+j-1])

    return dp[m][n]
```

Space-optimized (O(n)):

```
def isInterleave_optimized(s1, s2, s3):
    m, n = len(s1), len(s2)
    if m + n != len(s3):
        return False

    dp = [False] * (n + 1)
    dp[0] = True

    for j in range(1, n + 1):
        dp[j] = dp[j-1] and s2[j-1] == s3[j-1]

    for i in range(1, m + 1):
        dp[0] = dp[0] and s1[i-1] == s3[i-1]
        for j in range(1, n + 1):
            dp[j] = (dp[j] and s1[i-1] == s3[i+j-1]) or \
                    (dp[j-1] and s2[j-1] == s3[i+j-1])

    return dp[n]
```

Time Complexity

Time: $O(mn)$ — Fill $(m+1) \times (n+1)$ table
Space: $O(mn)$ standard, $O(n)$ optimized

Correctness Proof

Claim: $dp[i][j]$ = True iff $s_3[0:i+j]$ can be formed by interleaving $s_1[0:i]$ and $s_2[0:j]$.

Proof by strong induction on (i, j):

Base:
- $dp[0][0]$ = True (empty strings interleave to empty) - $dp[i][0]$: Only s_1, check if $s_1[0:i] = s_3[0:i]$ - $dp[0][j]$: Only s_2, check if $s_2[0:j] = s_3[0:j]$

IH: Assume correct for all (i', j') with $i' \leq i$ and $j' < j$, or $i' < i$.

Step: For $dp[i][j]$, $s_3[0:i+j]$ can be formed iff:

Case 1: Last char came from s_1
→ Need: $s_3[0:i+j-1]$ formed by interleaving $s_1[0:i-1]$ and $s_2[0:j]$
→ By IH: $dp[i-1][j]$ is correct
→ And: $s_1[i-1] = s_3[i+j-1]$

Case 2: Last char came from s_2
→ Need: $s_3[0:i+j-1]$ formed by interleaving $s_1[0:i]$ and $s_2[0:j-1]$
→ By IH: $dp[i][j-1]$ is correct
→ And: $s_2[j-1] = s_3[i+j-1]$

Algorithm checks both cases (OR), so $dp[i][j]$ is correct. □

Quick Reference

Complexity Summary

Problem	Time	Space	Pattern
Triangle Path	$O(n^2)$	$O(n)$	Grid DP
Coin Change	$O(nV)$	$O(V)$	1D DP
Matrix Chain	$O(n^3)$	$O(n^2)$	Interval DP
Edit Distance	$O(mn)$	$O(mn)$	2D Grid
LPS	$O(n^2)$	$O(n^2)$	Interval DP
SCS	$O(mn)$	$O(mn)$	Via LCS
Beautiful Seq	$O(n^2)$	$O(n^2)$	Via LPS
Subset Sum	$O(nt)$	$O(t)$	Knapsack variant
Knapsack	$O(nW)$	$O(W)$	2D→1D
LIS	$O(n^2)$ or $O(n \log n)$	$O(n)$	1D DP or Binary Search
Rod Cutting	$O(n^2)$	$O(n)$	Unbounded Knapsack
Max Subarray	$O(n)$	$O(1)$	Kadane's
Climbing Stairs	$O(n)$	$O(1)$	Fibonacci
House Robber	$O(n)$	$O(1)$	1D DP
Word Break	$O(n^2m)$	$O(n)$	1D DP
Partition	$O(n \cdot \text{sum})$	$O(\text{sum})$	Subset Sum
Egg Drop	$O(kn^2)$	$O(kn)$	2D DP

Pattern Recognition Guide

1. Sequential (1D Array)

When: Process elements left-to-right, decision at each position
Examples: Max subarray, climbing stairs, house robber
Template: $dp[i] = f(dp[i-1], dp[i-2], \ldots, arr[i])$

2. Two Sequences (2D Grid)

When: Align/compare two strings or arrays
Examples: LCS, Edit distance, SCS
Template: $dp[i][j] = f(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$

3. Intervals/Ranges

When: Combine subranges, try all split points
Examples: Matrix chain, palindrome problems
Template: $dp[i][j] = \min_{k=i}^{j-1} f(dp[i][k], dp[k+1][j])$

4. Knapsack Family

When: Items + capacity/target, include/exclude decisions
Examples: 0/1 knapsack, subset sum, partition
Template: $dp[i][w] = \max / \vee (dp[i-1][w], dp[i-1][w-w_i] + v_i)$

5. Special Patterns

- **Fibonacci-like:** Climbing stairs (sum of 2 previous)
- **Greedy + DP:** Activity selection, Huffman coding
- **Bitmask DP:** TSP, subset problems with $n \leq 20$

Master Theorem Cheat Sheet

For recurrence: $T(n) = aT(n/b) + f(n)$

Step 1: Compute $c = \log_b a$

Step 2: Compare $f(n)$ with n^c:

Comparison	Result
$f(n) = O(n^{c-\epsilon})$ (smaller)	$T(n) = \Theta(n^c)$
$f(n) = \Theta(n^c \log^k n)$ (equal)	$T(n) = \Theta(n^c \log^{k+1} n)$
$f(n) = \Omega(n^{c+\epsilon})$ (larger)	$T(n) = \Theta(f(n))$

Common values: - $\log_2 2 = 1$, $\log_2 4 = 2$, $\log_3 9 = 2$, $\log_4 2 = 0.5$

Examples: - $T(n) = 2T(n/2) + n \to c = 1$, $f(n) = n = \Theta(n^1) \to$ **Case 2** $\to O(n \log n)$ - $T(n) = 4T(n/2) + n \to c = 2$, $f(n) = n < n^2 \to$ **Case 1** $\to O(n^2)$ - $T(n) = 2T(n/2) + n^2 \to c = 1$, $f(n) = n^2 > n \to$ **Case 3** $\to O(n^2)$

Key Formulas

Summations: - Arithmetic: $\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = O(n^2)$ - Geometric: $\sum_{i=0}^{k} r^i = \frac{r^{k+1}-1}{r-1}$ - Harmonic: $\sum_{i=1}^{n} \frac{1}{i} = O(\log n)$

Fibonacci:

$$F(n) = \frac{\phi^n - \psi^n}{\sqrt{5}} \text{ where } \phi = \frac{1+\sqrt{5}}{2} \approx 1.618$$

Binomial:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Optimization Techniques

Space Optimization

1. **Sliding Window:** If $dp[i]$ depends only on $dp[i-1], dp[i-2]$, use 2 variables
   - Example: Fibonacci, climbing stairs
2. **1D from 2D:** If row i depends only on row $i-1$, use 1D array
   - Example: Knapsack, triangle path
3. **Backward Iteration:** Essential for space-optimized knapsack
   - Prevents using updated values in same iteration

Time Optimization

1. **Binary Search:** LIS $O(n^2) \to O(n \log n)$
2. **Precomputation:** Store prefix sums, factorials
3. **Pruning:** Skip impossible states early

Proof Templates

Induction Proof

```
Claim: [Statement]

Proof by [Simple/Strong] Induction on [variable]:

Base Case: [n=0 or n=1]
  [Show statement holds]


Inductive Hypothesis:
  Assume [statement] holds for n=k (or all n   k if strong)

Inductive Step: Show holds for n=k+1
  [Manipulate using IH]
  [Show equals/satisfies desired property]


Conclusion: By induction, [statement] holds for all n   n .
```

Loop Invariant Proof

```
Loop Invariant: [Property that remains true]

Initialization: [True before first iteration]
Maintenance: [If true before iteration k, remains true after]
Termination: [When loop ends, invariant + condition → correctness]

Conclusion: Algorithm is correct.
```

Quiz Strategy

Before the Quiz

1. Master recurrence writing - define state first!
2. Know Master Theorem cold (3 cases + $\log_b a$ computation)
3. Practice induction proofs (base, IH, step, conclusion)
4. Understand difference: LCS vs Substring, Bellman-Ford vs Dijkstra
5. Memorize key complexities: $O(n^2)$ vs $O(n \log n)$ vs $O(nt)$

During the Quiz

1. **Read carefully:** Max/min/count/exists?
2. **Define state precisely:** What does $dp[i][j]$ mean?
3. **Work small example:** Fill 3×3 table manually
4. **Check boundaries:** What if $i = 0$ or $j = 0$?
5. **State complexity clearly:** Count loops, mention optimization

Common Mistakes to Avoid

- Wrong iteration order in space-optimized DP (iterate backwards!)
- Confusing "ending at i" vs "up to i"
- Off-by-one in interval DP (inclusive vs exclusive)
- Forgetting base cases
- Incorrect Master Theorem case selection

Final Checklist

I can write recurrence relations from problem statements
I can solve recurrences using Master Theorem
I understand all 3 Master Theorem cases + when each applies
I can compute $\log_b a$ quickly and compare $f(n)$ with $n^{\log_b a}$
I know when to use 1D vs 2D DP
I can fill DP tables manually for small examples
I can analyze time and space complexity
I understand space optimization (sliding window, backward iteration)
I can write induction proofs (base, IH, step, conclusion)
I know the difference: LCS vs Longest Common Substring
I understand knapsack variants (0/1, unbounded, subset sum)
I've practiced enough to feel confident!

Remember: DP is about recognizing patterns and optimal substructure. Understand the "why" behind recurrences, not just memorization. Good luck on your quiz!